PREPGRADE

AI-Powered Assessment, CBT & Examination Preparation Platform

Complete Product, Technical, Business & Implementation Proposal

Prepared for: PrepGrade Founding Team

Project: PrepGrade

Industry: Education Technology / SaaS / Assessment Technology

Primary Market: Nigeria

Expansion Market: Africa

Business Model: B2B2C SaaS + Institutional Licensing + Student Subscriptions

Initial Strategy: MVP → Pilot Schools → Paying Customers → Retention → Expansion

1. EXECUTIVE SUMMARY

PrepGrade is a technology platform designed to help students, teachers, tutorial centres and educational institutions conduct, manage and analyse examinations and continuous assessments digitally.

The platform will initially focus on:

- JAMB preparation
- WAEC preparation
- NECO preparation
- Computer-Based Tests (CBT)
- Custom school examinations
- Question banks
- Automatic grading
- Student performance analytics

- Teacher dashboards
- Institution dashboards
- Diagnostic assessment
- Weak-topic identification
- Progress tracking

The long-term objective is to build **Nigeria's leading assessment intelligence platform**, beginning with examination preparation and institutional CBT and eventually expanding into a broader African assessment infrastructure.

PrepGrade will not attempt to compete immediately with large school-management platforms by offering accounting, fees, transport, hostel management, payroll and every other administrative feature.

Instead:

PrepGrade will own the assessment layer.

The company's central question will be:

"How well does this student actually understand what they are being tested on, and what should they do next?"

This creates a focused product with a clear value proposition.

2. THE VISION

Vision

To become Africa's most trusted digital assessment and examination intelligence platform.

Mission

To make high-quality assessment, examination preparation and academic performance intelligence accessible to every student, teacher and educational institution.

Long-Term Vision

PrepGrade should eventually enable a student to move through the entire assessment journey:

Learn → Practice → Test → Analyse → Improve → Retest → Master

The platform should eventually know:

- what the student has studied;
- what the student has attempted;
- what the student understands;
- where the student struggles;
- what topics are improving;
- what topics remain weak;
- how the student compares with previous performance;
- what the student should practise next.

3. THE PROBLEM

Nigeria has a huge examination and assessment market.

Students prepare for examinations such as:

- JAMB/UTME
- WAEC
- NECO
- BECE
- school examinations
- tutorial-centre examinations
- entrance examinations
- scholarship examinations
- professional examinations

However, assessment is often fragmented.

A typical student may use:

- textbooks;
- WhatsApp groups;
- physical past questions;
- random websites;
- mobile applications;
- tutorial centres;
- school examinations;
- separate CBT applications.

The problem is not simply the absence of questions.

The bigger problem is the absence of **connected assessment intelligence**.

A student can answer 500 questions and still not know:

"Why am I repeatedly failing Mathematics?"

A teacher can conduct an examination but spend hours manually analysing results.

A tutorial centre can have hundreds of students but lack a clear system for identifying:

- weakest students;
- strongest students;
- weakest subjects;
- weakest topics;
- questions most frequently missed;
- students at risk of poor performance.

PrepGrade aims to solve this.

4. THE OPPORTUNITY

PrepGrade operates at the intersection of:

- EdTech
- SaaS
- Assessment Technology
- AI
- CBT
- Data Analytics
- Examination Preparation
- Institutional Software

The Nigerian education technology market is competitive, but this also validates that schools and education providers are already purchasing digital education software.

For example, Edves currently offers school administration, assessment, CBT, AI pedagogy and examination-related capabilities and reports serving thousands of schools.

This means PrepGrade should not enter the market with:

"We have another school management system."

It should enter with:

"We are the assessment and examination intelligence layer for your institution."

5. PRODUCT POSITIONING

PrepGrade is NOT initially:

- a school ERP;
- accounting software;
- payroll software;
- hostel-management software;
- transport-management software;
- generic LMS;
- social network for students.

PrepGrade IS:

An Assessment & Examination Intelligence Platform.

Its primary responsibilities are:

Create → Deliver → Grade → Analyse → Improve

6. TARGET CUSTOMERS

PrepGrade should initially target five customer segments.

Segment 1 — Tutorial Centres

This should be one of the first markets.

Examples:

- JAMB tutorial centres
- WAEC tutorial centres
- NECO tutorial centres
- private coaching centres
- examination academies

Why?

They already have:

- students;
- teachers;
- examinations;
- recurring cohorts;
- strong need for results;
- willingness to pay for tools that improve performance.

7. SEGMENT 2 — Secondary Schools

Target:

- private secondary schools;
- Islamic schools;
- international schools;
- examination-focused schools;
- schools preparing students for WAEC/NECO/JAMB.

The school can use PrepGrade for:

- weekly tests;
- monthly tests;
- mock examinations;
- terminal examinations;
- revision;
- CBT;
- class assessments.

8. SEGMENT 3 — Teachers

Teachers can use PrepGrade to:

- create assessments;
- select questions;
- assign tests;

- monitor students;
- analyse results;
- identify weak topics.

9. SEGMENT 4 — Students

Students can use PrepGrade to:

- practise;
- take mock examinations;
- receive instant results;
- identify weak topics;
- track progress;
- repeat tests;
- prepare for national examinations.

10. SEGMENT 5 — Educational Organisations

Long term:

- NGOs;
- education programmes;
- state education initiatives;
- scholarship programmes;
- tutorial networks;
- private education groups;
- examination-preparation organisations.

These could become enterprise customers.

11. THE CORE PRODUCT

PrepGrade should be built around six major modules.

MODULE 1 — Question Bank

The foundation of the platform.

Teachers/institutions can organise questions by:

- examination;
- subject;
- topic;
- subtopic;
- difficulty;
- year;
- question type;
- learning objective.

Example:

JAMB

→ Mathematics

→ Algebra

→ Quadratic Equations

→ Medium

→ Objective

12. MODULE 2 — Exam Builder

Teachers create examinations using a visual exam builder.

Example:

Mathematics Mock Examination

Questions: 60

Time: 90 minutes

Difficulty:

- Easy: 20%
- Medium: 50%
- Hard: 30%

The teacher can:

- select questions manually;
- randomise questions;
- randomise options;
- set time;
- set attempts;
- configure scoring;
- schedule the exam.

13. MODULE 3 — CBT ENGINE

This is one of PrepGrade's most important technical components.

The CBT engine should support:

- countdown timer;
- question navigation;
- next/previous;
- mark for review;
- answered/unanswered indicators;
- automatic submission;
- randomised questions;
- randomised options;
- attempt control;
- instant grading;
- network recovery;
- exam-session persistence.

A key requirement:

A temporary network interruption should not automatically destroy the student's examination.

The system should progressively save exam state.

14. MODULE 4 — RESULTS ENGINE

After an examination, PrepGrade should automatically calculate:

- score;
- percentage;
- grade;
- time spent;
- correct answers;
- incorrect answers;
- unanswered questions;
- subject performance;
- topic performance.

Example:

Topic	Questions	Correct	Accuracy
Algebra	15	12	80%
Geometry	15	8	53%
Statistics	10	9	90%
Probability	10	4	40%

Instead of simply saying:

Score: 66%

PrepGrade says:

Your strongest area is Statistics. Your weakest area is Probability.

That is the beginning of assessment intelligence.

15. MODULE 5 — STUDENT ANALYTICS

Every student should eventually have a personal performance profile.

Student Dashboard

The student sees:

Overall Performance

78%

Questions Attempted

1,248

Average Accuracy

74%

Tests Completed

32

Strongest Subject

English

Weakest Subject

Mathematics

Weakest Topic

Probability

Current Trend

↑ Improving

The system should make progress visible.

16. MODULE 6 — INSTITUTION DASHBOARD

The institution should see aggregate performance.

Example:

School Performance

Students: 642

Tests conducted: 87

Average score: 68%

Students improving: 71%

Students at risk: 12%

Weakest subject: Mathematics

Weakest topic: Algebra

The administrator should be able to drill down:

School → Class → Subject → Topic → Student

This is where PrepGrade becomes significantly more valuable than a simple CBT application.

17. TEACHER DASHBOARD

Teachers should see:

My Classes

SS3A — 42 students

SS3B — 38 students

SS2A — 45 students

Performance

Average score: 72%

Highest: 94%

Lowest: 31%

Weak Topics

1. Probability
2. Trigonometry
3. Algebra

The teacher can then create a targeted revision test.

18. THE DIFFERENTIATOR

PrepGrade should eventually have a powerful feature:

Diagnostic Mastery Engine

Instead of treating every question independently, PrepGrade analyses patterns.

For example:

A student gets:

- Question 3 wrong;
- Question 12 wrong;
- Question 18 wrong;
- Question 29 wrong.

The system detects that all four questions test:

Simultaneous Equations

It can therefore identify:

Possible mastery gap: Simultaneous Equations

The student is then recommended:

1. Review concept.
2. Attempt 10 beginner questions.
3. Attempt 10 intermediate questions.
4. Take mastery test.
5. Reassess.

This transforms PrepGrade from:

CBT software

into:

Assessment intelligence software.

19. AI FEATURES

AI should NOT be the first thing built.

AI should be introduced after the core assessment system works.

Phase 1 AI

Teacher AI Assessment Generator.

Teacher enters:

SS2 Mathematics — Quadratic Equations

PrepGrade generates draft questions.

The teacher reviews and approves them.

Important:

AI generates. Teacher approves.

AI should not automatically publish educational content without human review.

20. FUTURE AI FEATURES

AI Question Generator

Generate questions based on:

- subject;
- topic;
- difficulty;
- examination style;
- learning objective.

AI Question Explanation

After answering incorrectly:

"Why is this answer wrong?"

AI Weakness Coach

"You have struggled with probability in your last five tests."

AI Study Plan

"Spend 30 minutes today on Probability and 20 minutes on Algebra."

AI Exam Predictor

Eventually, using legitimate and ethically obtained performance data:

Identify areas where a student may require more preparation.

Predictions should be presented as preparation guidance, not guaranteed examination outcomes.

21. MULTI-TENANT ARCHITECTURE

PrepGrade should be designed as a true SaaS platform.

One infrastructure can support:

- School A
- School B
- Tutorial Centre C
- Teacher D
- Institution E

while keeping their data logically isolated.

Tenant hierarchy

Platform

↓

Institution

↓

Campus

↓

Classes

↓

Teachers

↓

Students

↓

Assessments

↓

Results

22. USER ROLES

Super Admin

Controls:

- platform;
- institutions;
- subscriptions;
- system configuration;
- billing;
- support;
- analytics.

Institution Admin

Controls:

- institution;
- teachers;

- students;
- exams;
- classes;
- reports.

Teacher

Controls:

- classes;
- questions;
- exams;
- assignments;
- results.

Student

Can:

- practise;
- take examinations;
- view results;
- track progress.

23. MVP — WHAT WE ACTUALLY BUILD FIRST

This is extremely important.

Do NOT build everything.

The first MVP should contain only:

Institution

- create institution;
- institution dashboard;
- student management;
- teacher management.

Teacher

- create question;
- question bank;
- create exam;
- assign exam;
- view results.

Student

- login;
- dashboard;
- take exam;
- timer;
- submit exam;
- view score.

System

- automatic grading;
- basic analytics;
- exam history.

That is enough.

24. WHAT WE DO NOT BUILD IN MVP

Do NOT initially build:

- AI tutor;
- advanced AI chatbot;
- mobile applications;
- parent application;
- payment gateway;
- payroll;
- accounting;
- transport;
- hostel;
- biometric attendance;
- complex recommendation engine;

- social network;
- video courses;
- marketplace;
- blockchain;
- cryptocurrency;
- unnecessary integrations.

The objective is:

Prove the core product before expanding it.

25. MVP SUCCESS CRITERIA

The MVP should not be considered successful simply because the software works.

It is successful if real users use it.

Target:

Pilot

3–5 institutions.

Users

300–1,000 students.

Teachers

20–50.

Assessments

100+.

Student completion

60% of assigned assessments.

Institution retention

At least 60–70% of pilot institutions continue using it after the initial pilot.

Paid conversion

At least 2–3 institutions agree to pay.

These are internal targets, not guaranteed outcomes.

26. THE DEVELOPMENT STRATEGY

The company should follow:

BUILD → MEASURE → LEARN

Not:

BUILD → SPEND → HOPE

27. PHASE 0 — CUSTOMER DISCOVERY

Duration:

2–4 weeks

Cost:

Very low

Before building extensively, interview:

- 10 school owners;
- 10 teachers;
- 10 tutorial-centre operators;
- 20 students.

Ask:

1. How do you currently conduct exams?
2. What software do you use?
3. What is frustrating?
4. How much time does result processing take?
5. How often do you conduct CBT?
6. How do you identify weak students?
7. Would you pay for a better solution?
8. What would make you switch?
9. What would stop you from using it?
10. What would you consider essential?

Do not sell the vision.

Understand the problem.

28. PHASE 1 — PROTOTYPE

Duration:

2–4 weeks

Create clickable UI prototypes.

Recommended screens:

- landing page;
- login;
- institution dashboard;
- teacher dashboard;
- student dashboard;
- question bank;
- exam builder;
- CBT interface;
- results;
- analytics.

At this stage, you can test the product without building the complete backend.

29. PHASE 2 — MVP DEVELOPMENT

Duration:

6–10 weeks

Build only the agreed MVP.

Recommended stack:

Frontend

Next.js / React

Backend

Supabase or a scalable backend architecture.

Database

PostgreSQL

Authentication

Supabase Auth / Clerk / equivalent.

Hosting

Cloud infrastructure with scalable deployment.

Analytics

Product analytics + internal database analytics.

Payments

Add only after the pilot demonstrates willingness to pay.

30. SECURITY

Security must be designed from day one.

PrepGrade will handle student data.

Therefore, security cannot be an afterthought.

Implement:

- secure authentication;
- role-based access control;
- tenant isolation;
- encrypted transport;
- secure password handling;
- audit logs;
- backups;
- access logging;
- secure APIs;
- least-privilege access;
- incident response procedures.

Nigeria's NDPC specifically emphasises data protection responsibilities around personal data, and schools handle data relating to young people.

Therefore:

Privacy and security should become part of PrepGrade's competitive advantage.

31. DATA OWNERSHIP

PrepGrade should clearly define:

Institution data

Owned/controlled according to the contractual and legal framework agreed with the institution.

Student data

Handled according to applicable data-protection law and institutional agreements.

PrepGrade intellectual property

The software, platform architecture, algorithms and proprietary technology remain PrepGrade's intellectual property.

Legal agreements should be reviewed by qualified Nigerian counsel before commercial deployment.

32. PILOT STRATEGY

Do not immediately attempt to sell to 1,000 schools.

Start with:

3–5 schools/centres.

Ideal pilot customers:

- technologically open;
- decision-maker accessible;
- 100–500 students;
- frequent examinations;
- motivated teachers;
- willing to provide feedback.

33. PILOT OFFER

Example:

Founding Pilot Programme

Duration: 60–90 days

Includes:

- institution account;
- teacher accounts;
- student accounts;
- CBT;
- question bank;
- exam builder;
- results;

- basic analytics;
- onboarding;
- direct support.

The pilot may be free or heavily discounted.

But there should be a clear transition:

Pilot → Paid Subscription

Free users are useful for testing.

Paying customers prove the business.

34. CUSTOMER ACQUISITION

Initial sales should be founder-led.

Do not spend heavily on advertisements.

Approach schools directly.

Channels:

- WhatsApp;
- LinkedIn;
- Facebook;
- personal networks;
- education associations;
- tutorial centres;
- school visits;
- teacher communities;
- referrals.

35. SALES MESSAGE

Do not say:

"We built a modern EdTech platform."

Say:

"PrepGrade helps your school conduct CBT examinations, automatically grade students and show teachers exactly where students are struggling."

The second message communicates value.

36. PRICING STRATEGY

Do not lock pricing permanently before pilot feedback.

Test several models.

Model A — Institution Subscription

Example:

₦50,000–₦150,000/month depending on institution size and features.

Model B — Annual School Plan

Example:

₦300,000–₦1,500,000/year depending on student count and functionality.

Model C — Per Student

Example:

₦1,000–₦5,000/student/year.

Model D — Tutorial Centre

Example:

Monthly subscription based on active students.

The exact price should be determined through customer interviews and pilot willingness-to-pay testing.

37. FREEMIUM STRATEGY

Potential free tier:

- limited students;
- limited exams;
- limited question storage.

Paid tier:

- unlimited/expanded assessments;
- advanced analytics;
- branded institution;
- larger student capacity;
- reports;
- advanced features.

However:

Do not make the free plan so generous that institutions never need to upgrade.

38. RETENTION STRATEGY

Acquisition is not enough.

PrepGrade must become part of the institution's weekly routine.

The goal is:

Make PrepGrade operationally useful every week.

Examples:

Monday:

Teacher creates assessment.

Tuesday:

Students take assessment.

Wednesday:

Teacher reviews analytics.

Thursday:

Weak-topic test assigned.

Friday:

Students retake.

Monthly:

Institution reviews performance.

This creates recurring product usage.

39. KEY PRODUCT LOOP

The most important loop is:

Assessment

↓

Result

↓

Diagnosis

↓

Recommendation

↓

Practice

↓

New Assessment

↓

Improvement

↓

More Assessment

This is the engine that can create retention.

40. PRODUCT ROADMAP

Version 0.1 — MVP

- authentication;
- institutions;
- teachers;
- students;
- question bank;
- exam builder;
- CBT;
- auto-grading;
- results.

Version 0.2 — Pilot

Add:

- better analytics;
- class rankings;
- exam history;
- performance reports;
- question import;
- institution branding;
- improved CBT reliability.

Version 0.3 — Commercial

Add:

- subscriptions;
- billing;
- advanced analytics;
- institution reports;
- teacher reports;
- student performance profiles.

Version 1.0

Add:

- diagnostic engine;
- mastery tracking;
- AI question generation;
- AI explanations;
- intelligent recommendations.

Version 2.0

Add:

- adaptive practice;
- advanced AI;
- multilingual support;
- mobile apps;
- API;
- enterprise integrations.

41. HAUSA AND NORTHERN NIGERIA OPPORTUNITY

PrepGrade should eventually support:

- English;
- Hausa;

- potentially Yoruba;
- potentially Igbo.

However, multilingual support should not delay the MVP.

A smart strategy would be:

MVP

English.

Pilot

English + selected Hausa accessibility.

Expansion

Hausa-first features where market evidence supports them.

This could provide PrepGrade with a meaningful competitive advantage in Northern Nigeria.

42. CONTENT STRATEGY

Content is one of the biggest risks.

PrepGrade must not simply copy copyrighted examination materials.

The platform should use:

- original questions;
- teacher-created questions;
- licensed content;
- properly permitted materials;
- public-domain materials where applicable.

For national-examination preparation, rights and licensing must be treated as a serious business issue.

43. QUESTION QUALITY SYSTEM

Every question should eventually have metadata:

Question ID
Exam
Subject
Topic
Subtopic
Difficulty
Question Type
Correct Answer
Explanation
Source
Copyright/License Status
Created By
Reviewed By
Version

This becomes a valuable proprietary content infrastructure.

44. CONTENT QUALITY CONTROL

AI-generated questions must not automatically enter the production question bank.

Process:

AI Draft

↓

Teacher Review

↓

Academic Review

↓

Quality Check

↓

Publish

↓

Performance Monitoring

If students consistently report that a question is ambiguous, PrepGrade should flag it.

45. BUSINESS MODEL

PrepGrade should eventually have multiple revenue streams.

1. Institutional SaaS

Recurring subscription.

2. Tutorial Centre SaaS

Recurring subscription.

3. Student Premium

Advanced student features.

4. Enterprise Licensing

Large institutions and education organisations.

5. Assessment API

Other educational applications can use PrepGrade's assessment infrastructure.

6. White-Label

Large institutions can deploy branded versions.

7. Analytics

Advanced institutional performance analytics.

The company should initially focus on **one or two revenue streams**, not seven.

46. COMPETITIVE STRATEGY

The market already contains school-management platforms with administration, assessment and CBT capabilities. Edves, for example, currently combines school management with CBT, assessment and AI-powered pedagogy.

Therefore PrepGrade's competitive strategy should be:

Do not compete on:

"more features."

Compete on:

better assessment experience.

Position:

"The assessment infrastructure for Nigerian schools and examination centres."

47. COMPETITIVE MOAT

PrepGrade's moat should eventually consist of:

1. Assessment Data

Large volumes of legitimate performance data.

2. Question Intelligence

High-quality structured question bank.

3. Institutional Relationships

Schools and tutorial centres integrated into the platform.

4. Teacher Workflow

Teachers create and manage assessments through PrepGrade.

5. Student History

Longitudinal student performance profiles.

6. Diagnostic Algorithms

Understanding why students are failing.

7. Brand

Trust among Nigerian educators.

8. Distribution

School networks and education partnerships.

48. THE MOST IMPORTANT METRICS

Do not obsess over downloads.

Track:

Acquisition

- number of institutions contacted;
- demos;
- pilots;
- registrations.

Activation

- institutions creating first exam;
- teachers creating first assessment;
- students taking first exam.

Engagement

- exams per institution;
- assessments per student;

- weekly active users;
- monthly active users.

Learning

- average accuracy;
- improvement;
- mastery progression.

Retention

- institutions active after 30 days;
- 60 days;
- 90 days.

Revenue

- MRR;
- ARR;
- average revenue per institution;
- conversion rate;
- churn.

49. NORTH STAR METRIC

PrepGrade should eventually use:

Meaningful Assessments Completed

Not:

- app downloads;
- registered users;
- website visits.

A meaningful assessment is one that:

1. is completed;

2. produces valid results;
3. contributes to the student's performance profile.

This metric connects usage directly to the product's mission.

50. 12-MONTH EXECUTION PLAN

MONTH 1

Discovery

- interview 30 potential users;
- identify top problems;
- analyse competitors;
- define MVP;
- create product requirements document.

Deliverable:

Validated MVP specification.

MONTH 2

Prototype

Create:

- landing page;
- student dashboard;
- teacher dashboard;
- institution dashboard;
- exam builder;
- CBT interface;
- results.

Deliverable:

Clickable prototype.

MONTH 3

MVP Development

Build:

- authentication;
- database;
- institution;
- teachers;
- students;
- question bank;
- exam engine.

MONTH 4

MVP Completion

Complete:

- CBT;
- auto-grading;
- results;
- basic analytics;
- security;
- testing.

MONTH 5

Pilot

Launch with:

3–5 institutions.

Observe everything.

MONTH 6

Improve

Fix:

- usability;
- bugs;
- performance;
- onboarding;
- exam workflow;
- reporting.

MONTH 7

Paid Pilot

Convert the best pilot customers into paying customers.

Target:

2–5 paying institutions.

MONTH 8

Product-Market Fit Testing

Measure:

- retention;
- usage;
- willingness to pay;
- churn;
- teacher engagement.

MONTH 9

Commercial Launch

Start structured sales.

Target:

10–20 institutions.

MONTH 10

Assessment Intelligence

Introduce:

- topic analytics;
- student weakness detection;
- advanced reporting.

MONTH 11

AI

Introduce:

Teacher AI Question Generator

with mandatory teacher review.

MONTH 12

Strategic Review

Evaluate:

- revenue;

- active institutions;
- retention;
- student usage;
- product quality;
- acquisition cost;
- churn.

Then decide:

Scale / Pivot / Continue Testing / Stop

51. LEAN BUDGET STRATEGY

The philosophy is:

Do not spend ₦10 million trying to discover whether someone wants a ₦500,000 product.

The first phase should be extremely lean.

Spend primarily on:

- domain;
- hosting;
- development;
- essential software;
- design;
- testing;
- customer acquisition;
- legal/compliance basics.

Avoid:

- expensive office;
- large team;
- unnecessary branding;
- expensive advertising;
- full mobile applications;
- unnecessary AI infrastructure;
- large server commitments.

52. TEAM STRUCTURE

Initially:

Founder / CEO

Responsibilities:

- vision;
- sales;
- partnerships;
- product;
- customer discovery.

Technical Lead

Responsibilities:

- architecture;
- development;
- security;
- deployment.

Product/UI Designer

Can initially be part-time/freelance.

Academic/Content Specialist

Initially part-time.

Customer Success

Initially handled by founder.

Do not hire ten people before revenue.

53. FOUNDER RESPONSIBILITIES

The founder should spend the first year approximately:

30%

Customer acquisition.

25%

Product.

20%

Customer research/support.

15%

Business operations.

10%

Strategy.

The founder should not spend 90% of their time coding.

A company cannot scale if the founder becomes the only developer, salesperson, support person and administrator.

54. GO-TO-MARKET STRATEGY

Start geographically narrow.

For example:

Phase 1

Northern Nigeria / Abuja / nearby accessible market.

Phase 2

Major Nigerian cities.

Phase 3

Nationwide Nigeria.

Phase 4

West Africa.

Phase 5

Africa.

The objective is not:

"Launch everywhere."

It is:

Dominate one clearly defined niche first.

55. FIRST CUSTOMER PROFILE

The ideal first customer is not necessarily the biggest school.

It is:

A school or tutorial centre with a real examination problem and a decision-maker who is willing to experiment.

A 300-student institution that loves PrepGrade is more valuable initially than a 10,000-student institution that takes six months to approve a pilot.

56. PILOT CUSTOMER SUCCESS PLAN

Each pilot should have:

Day 1

Onboarding.

Day 7

First assessment.

Day 14

Usage review.

Day 30

Performance report.

Day 45

Teacher feedback.

Day 60

Renewal conversation.

Day 90

Paid conversion/expansion.

57. CUSTOMER SUCCESS QUESTIONS

Every month ask:

1. What do you use most?
2. What do you not use?
3. What frustrates you?
4. What saves you the most time?
5. What would make you recommend PrepGrade?
6. What feature do you need next?
7. Would you continue paying?
8. Why?

This feedback becomes the product roadmap.

58. RISK MANAGEMENT

Risk 1 — Low adoption

Solution:

Make onboarding extremely simple.

Risk 2 — Teachers dislike technology

Solution:

Teacher-first UX.

Risk 3 — Poor question quality

Solution:

Academic review system.

Risk 4 — Copyright problems

Solution:

Content provenance and licensing controls.

Risk 5 — Competition

Solution:

Focus deeply on assessment intelligence.

Risk 6 — Infrastructure failure

Solution:

Cloud redundancy, monitoring, backups and disaster recovery.

Risk 7 — Data breach

Solution:

Security architecture and privacy-by-design.

Risk 8 — Students bypass the system

Solution:

Make the platform genuinely useful rather than relying entirely on restrictions.

Risk 9 — Founder builds too many features

Solution:

Strict MVP roadmap.

Risk 10 — Spending too much too early

Solution:

Milestone-based funding.

59. FUNDING PRINCIPLE

PrepGrade should follow:

Revenue Before Major Funding

if possible.

Stage 1:

Founder-funded MVP.

Stage 2:

Pilot.

Stage 3:

Early revenue.

Stage 4:

Product-market fit.

Stage 5:

External funding only if it accelerates a proven model.

Funding should accelerate the machine.

It should not be used to discover whether the machine works.

60. MILESTONE-BASED CAPITAL

Stage 1

~~₦~~0–~~₦~~500k range:

Validate problem and prototype.

Stage 2

Lean MVP:

Spend only what is required to launch.

Stage 3

Pilot:

Spend based on actual usage.

Stage 4

Revenue:

Reinvest revenue into product.

Stage 5

Scale:

Consider significant external capital only when there is evidence of:

- demand;
- retention;
- recurring revenue;
- strong unit economics;
- scalable acquisition.

The actual amounts should be determined from the MVP scope and founder resources rather than arbitrary spending targets.

61. INVESTOR READINESS

Do not approach investors saying:

"We have an idea for an EdTech platform."

Approach investors with:

"We have X institutions, X students, X monthly assessments, X% retention, ₦X MRR, X% monthly growth and evidence that institutions are willing to pay."

That is a business.

62. POSSIBLE INVESTMENT STORY

Eventually:

Problem

Assessment in African education remains fragmented.

Solution

PrepGrade provides digital assessment infrastructure and intelligence.

Market

Schools, tutorial centres, students and education organisations.

Traction

Paying institutions and recurring assessments.

Moat

Assessment data + institutional workflows + question intelligence + diagnostic algorithms.

Expansion

Nigeria → West Africa → Africa.

63. FIVE-YEAR ROADMAP

YEAR 1 — VALIDATE

Goal:

Prove demand.

Targets:

- MVP;
- 5–20 institutions;
- first paying customers;
- recurring usage.

YEAR 2 — PRODUCT-MARKET FIT

Goal:

Build a repeatable business.

Focus:

- stronger CBT;
- analytics;
- subscriptions;
- institutional sales;
- retention;
- customer success.

YEAR 3 — SCALE

Goal:

Become a recognised Nigerian assessment platform.

Expand:

- nationwide;
- larger school networks;
- tutorial centres;
- partnerships;
- AI assessment.

YEAR 4 — INTELLIGENCE

Goal:

Build assessment intelligence.

Develop:

- mastery engine;
- adaptive testing;
- predictive analytics;
- AI teacher tools;
- personalised practice.

YEAR 5 — AFRICA

Goal:

Expand beyond Nigeria.

Potential markets:

- Ghana;
- Kenya;
- Uganda;
- Tanzania;
- other African education markets.

The expansion should follow evidence rather than geography for its own sake.

64. THE LONG-TERM PRODUCT

The ultimate PrepGrade platform could look like:

PrepGrade Assessment Cloud

Assessment

Create exams.

CBT

Deliver exams.

Question Intelligence

Manage questions.

Analytics

Understand performance.

AI

Generate and explain.

Mastery

Track knowledge.

Practice

Recommend questions.

Institution

Manage assessment workflows.

API

Allow other EdTech products to integrate.

This becomes infrastructure rather than simply another student app.

65. PREPGRADE API — LONG-TERM OPPORTUNITY

Eventually, third-party educational applications could integrate:

POST /assessments
POST /questions
POST /attempts
GET /results
GET /analytics
GET /mastery

A school-management platform could use PrepGrade for assessment.

A tutoring application could use PrepGrade for testing.

An NGO could use PrepGrade for programme evaluation.

A government education programme could use PrepGrade for large-scale assessment.

That is a much larger opportunity than one application.

66. BRAND POSITIONING

PrepGrade should communicate:

Smart

Assessment intelligence.

Reliable

Examinations must work.

Simple

Teachers should not need technical expertise.

Professional

Institutions should trust it.

Nigerian

Built around local examination realities.

Scalable

Designed for Africa.

67. POSSIBLE TAGLINES

Primary

Practice. Assess. Improve.

Alternative

Know Where You Stand. Know What to Improve.

Institutional

Turn Every Assessment Into Insight.

Student

Practice Smarter. Perform Better.

Enterprise

Assessment Intelligence for Modern Education.

68. THE PRODUCT PRINCIPLE

Every feature should answer at least one of these questions:

Does it help create an assessment?

Does it help deliver an assessment?

Does it help understand performance?

Does it help improve learning?

Does it help an institution operate its assessment process?

If the answer is no:

Do not build it yet.

69. THE FOUNDER'S DECISION FRAMEWORK

Before approving any feature, ask:

1.

Will customers pay because of it?

2.

Will it increase retention?

3.

Will it increase assessment usage?

4.

Does it differentiate PrepGrade?

5.

Can we build it efficiently?

6.

Does it create a long-term moat?

If the answer to most is "no":

Reject the feature.

70. THE MOST IMPORTANT BUSINESS LOOP

PrepGrade should create this cycle:

Acquire institution

↓

Onboard teachers

↓

Teachers create exams

↓

Students take exams

↓

Results generated

↓

Teachers discover weaknesses

↓

Teachers create targeted assessments

↓

Students improve

↓

Institution sees measurable value

↓

Institution renews

↓

Institution recommends PrepGrade

↓

New institution joins

That is the growth engine.

71. WHAT SUCCESS LOOKS LIKE

The first major victory is NOT:

100,000 registered students.

It is:

10 schools paying because PrepGrade has become important to their examination workflow.

Then:

50 institutions.

Then:

100.

Then:

1,000.

Scale should follow retention.

72. THE 10-SCHOOL TEST

Before serious expansion, achieve:

10 paying institutions

with:

- recurring monthly/annual payments;
- consistent assessment usage;
- strong retention;
- teacher adoption;
- student activity;
- measurable value.

If 10 institutions love the product, the company has something worth scaling.

If 10 institutions do not want it:

Do not spend millions scaling it.

Fix the product first.

73. FINAL STRATEGIC RECOMMENDATION

If I were managing PrepGrade from day one, I would follow this exact sequence:

STEP 1

Interview customers.

STEP 2

Validate the problem.

STEP 3

Design the smallest useful product.

STEP 4

Build the MVP.

STEP 5

Deploy to 3–5 institutions.

STEP 6

Watch real users.

STEP 7

Fix the biggest problems.

STEP 8

Convert pilots into paying customers.

STEP 9

Measure retention.

STEP 10

Build assessment intelligence.

STEP 11

Introduce AI.

STEP 12

Scale sales.

STEP 13

Build institutional partnerships.

STEP 14

Expand nationally.

STEP 15

Expand across Africa.

74. THE PREPGRADE COMPANY STRATEGY

The company should eventually be organised around four pillars:

1. ASSESS

Examinations and CBT.

2. ANALYSE

Performance intelligence.

3. IMPROVE

Personalised practice and recommendations.

4. SCALE

Institutional and enterprise infrastructure.

75. FINAL "IF I WERE THE FOUNDER" PLAN

I would NOT start by building a huge EdTech ecosystem.

I would start with one painful problem:

Schools and tutorial centres need a better way to conduct assessments and understand student performance.

I would build:

Question Bank + Exam Builder + CBT + Auto-Grading + Results + Basic Analytics.

Then I would find:

5 schools.

Then:

10 schools.

Then:

25 schools.

Then:

100 schools.

Only after seeing repeated usage would I invest heavily in:

- AI;
- adaptive learning;
- mobile apps;
- advanced analytics;
- national expansion.

The company would evolve from:

CBT software

to:

Assessment platform

to:

Assessment intelligence platform

to:

African assessment infrastructure.

76. THE PREPGRADE NORTH STAR

The ultimate goal is not to build another education app.

It is to build the infrastructure that answers:

What does this student know?

What does this student not know?

Why are they struggling?

What should they practise next?

Are they improving?

How can the teacher help?

How is the institution performing?

If PrepGrade can answer those questions accurately and consistently, it can become a significant education technology company.

77. FINAL EXECUTION PHILOSOPHY

DO NOT:

Build everything.

DO:

Build the smallest product that solves a painful problem.

DO NOT:

Spend millions first.

DO:

Validate first.

DO NOT:

Chase downloads.

DO:

Chase retention.

DO NOT:

Build AI because AI is fashionable.

DO:

Use AI where it produces measurable value.

DO NOT:

Try to beat every EdTech company.

DO:

Own the assessment layer.

DO NOT:

Expand before product-market fit.

DO:

Expand after evidence.

PREPGRADE

MVP → PILOT → PAYING CUSTOMERS → RETENTION → EXPANSION

Start small.

Prove value.

Earn revenue.

Build the moat.

Then scale.

Long-term objective:

BUILD AFRICA'S ASSESSMENT INTELLIGENCE INFRASTRUCTURE.

PREPGRADE — EXECUTION BLUEPRINT

1. PRODUCT ARCHITECTURE

PrepGrade should initially have **4 interfaces**:

A. Student App

/dashboard/student

Students can:

- See assigned exams
- Start CBT
- Continue interrupted exams
- Submit exams
- View results
- View performance history
- See weak subjects/topics
- Practise available questions

B. Teacher Dashboard

/dashboard/teacher

Teachers can:

- Create questions
- Manage question bank
- Create exams
- Assign exams
- Monitor students
- View results
- Analyse class performance

C. Institution Dashboard

/dashboard/school

School administrators can:

- Manage teachers
- Manage students
- Manage classes
- Create/manage exams
- View institution analytics
- Generate reports

D. Super Admin

/admin

PrepGrade's internal team controls:

- Institutions
- Users
- Subscriptions
- Questions
- System settings
- Support
- Platform analytics
- Audit logs

2. MVP USER FLOW

The simplest successful journey should be:

Institution

Register → Create school → Add teachers → Add students → Create class

↓

Teacher

Create question → Build exam → Assign exam

↓

Student

Login → See assigned exam → Start CBT → Answer → Submit

↓

System

Auto-grade → Generate result → Update analytics

↓

Teacher

See class performance → Identify weak areas

↓

Student

See personal performance → Practise again

That is the MVP.

3. MVP SCREEN LIST

Do not design 100 screens.

Start with approximately **20–25 high-quality screens**.

Public

1. Landing page
2. Pricing
3. Login
4. Registration
5. Forgot password

Institution

6. Institution dashboard
7. Students
8. Teachers
9. Classes
10. Institution settings

Teacher

11. Teacher dashboard
12. Question bank
13. Create question
14. Exam list
15. Exam builder
16. Exam details
17. Results
18. Class analytics

Student

19. Student dashboard
20. Exam instructions
21. CBT interface
22. Exam submission
23. Result
24. Performance analytics

Admin

25. Admin dashboard

That is enough for version 1.

4. DATABASE ARCHITECTURE

Use PostgreSQL.

A simplified structure:

users
├── institutions
├── teachers
└── students

institutions
├── campuses
├── classes

├── teachers
└── students

subjects
└── topics

questions
├── subjects
├── topics
└── question_options

exams
├── exam_questions
├── classes
└── assignments

attempts
└── answers

results
└── performance_metrics

More concretely:

users
institutions
institution_members
students
teachers
classes
subjects
topics
questions
question_options
exams
exam_questions
exam_assignments
exam_attempts
attempt_answers
results
performance_metrics
subscriptions
audit_logs

5. MULTI-TENANCY

This is extremely important.

Suppose:

Qurraa Academy

and

School B

both use PrepGrade.

School B must NEVER be able to access Qurraa Academy's data.

Every institution-owned record should therefore have an:

institution_id

For example:

students
---
id
institution_id
name
email
class_id

and:

exams
---
id
institution_id
teacher_id
title
duration
status

Then implement strict database-level access controls.

If using Supabase, PostgreSQL Row Level Security should be treated as a core security layer, not something added later.

6. ROLE-BASED ACCESS CONTROL

Use permissions such as:

SUPER_ADMIN

INSTITUTION_ADMIN

TEACHER

STUDENT

Example:

Teacher

Can:

CREATE question
READ own questions
CREATE exam
READ own classes
READ assigned results

Cannot:

DELETE institution
VIEW another institution
MODIFY billing
ACCESS system administration

Student

Can:

VIEW assigned exams
START exam
SUBMIT attempt
VIEW own results

Cannot:

VIEW another student's result
VIEW question bank
CREATE exam
VIEW teacher dashboard

7. QUESTION MODEL

Every question should eventually contain:

Question ID
Institution ID
Subject
Topic
Subtopic
Question Text
Question Type
Difficulty
Options
Correct Answer
Explanation
Marks
Source
Created By
Reviewed By
Status
Created At
Updated At

Question status:

DRAFT
UNDER_REVIEW
APPROVED
PUBLISHED
ARCHIVED

This is important because your future AI system will depend heavily on structured question data.

8. EXAM MODEL

An exam should contain:

Exam ID
Institution
Created By
Title
Description
Subject
Duration
Total Questions
Total Marks
Start Date
End Date
Attempts Allowed
Randomize Questions
Randomize Options
Status

Status:

DRAFT
SCHEDULED
ACTIVE
COMPLETED
ARCHIVED

9. CBT ENGINE

This is one area where I would spend serious engineering attention.

The CBT interface should have:

Header

- Exam name
- Remaining time
- Student name
- Question number

Main area

Question.

Options.

Navigation

1 2 3 4 5
6 7 8 9 10

Colours/status should represent:

- answered;
- unanswered;
- marked for review;
- current question.

10. EXAM AUTOSAVE

Never rely on a single final submission.

When a student answers:

Question 1 → save
Question 2 → save
Question 3 → save

The system should periodically persist the attempt.

If the student's internet temporarily fails:

The exam should recover instead of losing everything.

For higher reliability, you can eventually combine:

local temporary state + server synchronization.

11. TIMER SECURITY

Do not trust the student's browser clock.

The server should determine:

exam_start_time
exam_end_time

The browser displays the timer.

The server validates whether the attempt is still valid.

Otherwise, a technically knowledgeable student could manipulate the client-side timer.

12. EXAM SUBMISSION

When the student clicks:

SUBMIT EXAM

show:

You have answered 45/50 questions.

5 questions remain unanswered.

Are you sure you want to submit?

Then:

Cancel

Submit

Once submitted:

attempt.status = SUBMITTED

The student cannot continue that attempt unless the exam configuration explicitly allows another attempt.

13. RESULTS ENGINE

The result engine should calculate:

total_questions
correct_answers
wrong_answers
unanswered
score
percentage
time_used
accuracy

Example:

Questions: 50

Correct: 38
Wrong: 9
Unanswered: 3

Score: 76%

Accuracy: 80.85%

Then calculate topic performance.

14. BASIC ANALYTICS

For MVP, don't build complicated AI.

Use straightforward analytics.

Example:

Mathematics
76%

English
84%

Physics
61%

Chemistry
72%

Then:

Strongest

English — 84%

Weakest

Physics — 61%

Improvement

Previous: 55%

Current: 61%

Improvement:

+6 percentage points

This alone is useful.

15. THE FIRST "MAGIC" FEATURE

After the basic analytics works, build:

Weak Topic Detection

Example:

Student:

Physics

Overall: 61%

Topic performance:

Mechanics 82%
Waves 77%
Electricity 49%
Optics 73%

PrepGrade tells the student:

Electricity is currently your weakest Physics topic.

Then:

Practise Electricity.

This is the first step toward the future mastery engine.

16. TEACHER ANALYTICS

Teacher sees:

Class SS3A

Students: 42

Average: 68%

Highest: 94%

Lowest: 29%

Then:

Weakest topics

1. Probability — 43%
2. Trigonometry — 48%
3. Algebra — 52%

This gives the teacher an actionable answer:

"What should I teach again?"

That is far more valuable than merely showing grades.

17. INSTITUTION ANALYTICS

School administrator:

Total Students 642
Active Students 598
Exams Conducted 84
Average Score 69%

Then:

Subject Performance

Subject	Average
English	76%
Biology	73%
Chemistry	68%
Mathematics	61%
Physics	58%

Now the school can see where intervention is required.

18. PRODUCT DEVELOPMENT PRIORITY

Every feature should be classified:

P0 — Must have

Required for launch.

P1 — Important

Build immediately after MVP.

P2 — Useful

Build after traction.

P3 — Later

Do not build yet.

Example:

Feature	Priority
Authentication	P0
CBT	P0
Auto-grading	P0
Question Bank	P0
Exam Builder	P0
Basic Analytics	P0
AI Tutor	P3
Mobile App	P2
Parent App	P3
Payment Gateway	P1
Adaptive Learning	P2
AI Question Generator	P1
School Accounting	P3

19. 12-WEEK DEVELOPMENT SPRINT

WEEK 1 — Foundation

- Finalize requirements
- Create Git repository
- Set up project
- Configure database
- Configure authentication
- Establish design system
- Define architecture

Deliverable: Technical foundation.

WEEK 2 — Users & Institutions

Build:

- registration;
- login;
- institution creation;
- user roles;
- institution profile.

Deliverable: Users can enter the platform.

WEEK 3 — Students & Teachers

Build:

- student management;
- teacher management;
- classes;
- invitations;
- basic dashboards.

Deliverable: Institution can manage users.

WEEK 4 — Question Bank

Build:

- create question;
- edit question;
- delete/archive;
- subjects;
- topics;
- difficulty;
- options;
- correct answer.

Deliverable: Teacher can build question bank.

WEEK 5 — Exam Builder

Build:

- create exam;
- select questions;
- configure duration;
- configure attempts;

- assign classes;
- publish exam.

Deliverable: Teacher can create real exams.

WEEK 6 — CBT

Build:

- exam instructions;
- timer;
- question navigation;
- answers;
- mark for review;
- autosave;
- submission.

Deliverable: Student can complete a real CBT.

WEEK 7 — Results

Build:

- auto-grading;
- score;
- percentage;
- correct/wrong;
- exam history.

Deliverable: Student receives results.

WEEK 8 — Analytics

Build:

- student performance;
- class performance;

- subject performance;
- topic performance.

Deliverable: Teachers can understand results.

WEEK 9 — Security & Reliability

Test:

- permissions;
- tenant isolation;
- authentication;
- exam manipulation;
- concurrent users;
- network interruption;
- autosave;
- database security.

Deliverable: Pilot-ready system.

WEEK 10 — Pilot Preparation

Prepare:

- onboarding;
- documentation;
- demo environment;
- support channel;
- feedback system;
- analytics tracking.

Recruit:

3–5 pilot institutions.

WEEK 11 — Pilot

Deploy.

Observe.

Do not immediately add features.

Watch:

- where users get confused;
- where students drop;
- where teachers struggle;
- where exams fail;
- what users repeatedly request.

WEEK 12 — Fix & Evaluate

Prioritize:

Critical bugs → usability → reliability → requested features

Then decide:

Is PrepGrade ready for paid pilots?

20. PILOT DASHBOARD

Internally, you should maintain a dashboard like:

PILOT STATUS

Institutions: 5
Teachers: 27
Students: 483
Exams: 61
Attempts: 2,841
Completed: 2,632
Completion: 92%

Then:

Weekly Active Institutions
Weekly Active Teachers
Weekly Active Students
Assessments/Institution
Assessments/Student
Average Score
30-Day Retention

This is how you manage the company using data.

21. PRODUCT-MARKET-FIT TEST

At the end of the pilot, ask:

Would you continue using PrepGrade?

Would you pay for it?

How much?

What would make you stop?

What feature is most valuable?

Would you recommend it?

If customers say:

"It's nice."

That is not enough.

You want:

"We need this for our examinations."

That is a much stronger signal.

22. YOUR FIRST SALES TARGET

Do not target 1,000 schools.

Target:

10 paying institutions.

For each customer, document:

- number of students;
- number of teachers;
- number of exams;
- monthly usage;
- subscription;
- retention;
- complaints;
- feature requests.

Once you have 10 successful customers, you will understand the business much better.

23. SALES FUNNEL

Build this funnel:

100 institutions contacted
↓
40 respond
↓
20 demos
↓
10 pilots
↓
5 paying
↓
3 retained

These are **planning assumptions**, not guaranteed conversion rates.

Your actual numbers become your benchmark.

24. CUSTOMER ACQUISITION WITHOUT BIG AD SPEND

Initially use founder-led sales.

Every week:

Monday

Research 20 schools.

Tuesday

Contact 20.

Wednesday

Follow up.

Thursday

Conduct demos.

Friday

Visit/contact decision-makers.

Saturday

Onboard customers.

You need conversations more than advertisements.

25. THE DEMO

A PrepGrade demo should take approximately:

10 minutes

Show:

1.

Teacher creates exam.

2.

Student takes exam.

3.

System grades automatically.

4.

Teacher sees results.

5.

PrepGrade identifies weak topic.

Then say:

"This is what your teachers can see after every examination."

That is your product story.

26. CUSTOMER ONBOARDING

When a school joins:

Step 1

Create institution.

Step 2

Add teachers.

Step 3

Create classes.

Step 4

Import students.

Step 5

Create first assessment.

Step 6

Students take assessment.

Step 7

Teacher reviews analytics.

Your onboarding target:

First successful exam within 24 hours.

The faster a customer reaches value, the better your retention opportunity.

27. FUTURE AI ARCHITECTURE

Do not put an LLM everywhere.

AI should sit on top of structured assessment data.

Architecture:

Student Activity
↓
Assessment Engine
↓
Performance Data
↓
Analytics Engine

↓
Mastery Engine
↓
AI Layer
↓
Recommendations

AI without good assessment data will mostly produce impressive-looking features.

Good data + good algorithms + AI is much more powerful.

28. FUTURE MASTERY MODEL

Eventually each student can have:

Mathematics
├── Algebra 82%
├── Geometry 74%
├── Probability 43%
├── Statistics 89%
└── Calculus 61%

PrepGrade can classify:

MASTERED
STRONG
DEVELOPING
WEAK
CRITICAL

Then dynamically recommend practice.

29. LONG-TERM STUDENT PROFILE

Eventually:

Student
↓
Subjects

↓
Topics
↓
Questions
↓
Attempts
↓
Performance
↓
Mastery
↓
Recommendations

This becomes one of PrepGrade's strongest assets.

30. THE BIGGER VISION

Eventually PrepGrade could become:

PrepGrade OS for Assessment

Not a school-management OS.

An:

Assessment Operating System.

A school could use PrepGrade to manage the entire assessment lifecycle:

Question Creation
↓
Question Bank
↓
Exam Creation
↓
Exam Delivery
↓
Automatic Grading
↓
Analytics
↓

Weakness Detection
↓
Targeted Practice
↓
Retesting
↓
Mastery

That is the company I would build.

31. WHAT YOU SHOULD DO FIRST

If you are starting **this week**, don't code the entire platform.

Do these five things first:

1. Write the one-page MVP specification.

2. Create the 20–25 core screens.

3. Interview 30 potential users.

4. Find 3–5 pilot institutions.

5. Build only what those institutions actually need.

Your first objective isn't:

"Build PrepGrade."

It is:

"Prove that institutions need PrepGrade."

Once that is proven, you build faster and spend with confidence.

PREPGRADE — PRODUCT REQUIREMENTS DOCUMENT

Developer-Ready MVP Specification

Product: PrepGrade

Version: MVP v1.0

Product Type: Multi-tenant EdTech SaaS

Primary users: Institutions, teachers, students

Initial market: Nigeria

Core product: Assessment + CBT + Performance Analytics

1. PRODUCT DEFINITION

1.1 One-sentence description

PrepGrade is a multi-tenant assessment and CBT platform that enables schools and tutorial centres to create examinations, deliver them digitally, automatically grade students, and understand performance at student, class, subject and topic level.

1.2 MVP objective

The MVP must prove five things:

1. Teachers can create assessments without technical assistance.
2. Students can reliably complete CBT examinations.
3. PrepGrade can grade examinations accurately.
4. Teachers can understand student performance.
5. Institutions will continue using and eventually pay for the platform.

Everything else is secondary.

2. PRODUCT PRINCIPLES

Principle 1 — Simplicity

A teacher should be able to create an exam without training.

Principle 2 — Reliability

An examination platform cannot lose student answers.

Principle 3 — Security

One institution must never access another institution's data.

Principle 4 — Actionable analytics

Analytics should answer:

"What should I do next?"

not merely display numbers.

Principle 5 — Mobile-first

Many Nigerian users will access PrepGrade primarily from smartphones.

Principle 6 — Low-bandwidth awareness

The platform should work reasonably well under unstable internet conditions.

Principle 7 — Build only what has been validated

No unnecessary features during MVP.

3. MVP SCOPE

Included

Authentication

- Sign up
- Login
- Logout
- Password reset
- Role management

Institution

- Create institution
- Manage institution profile
- Manage teachers
- Manage students
- Manage classes

Teacher

- Question bank
- Create question
- Edit question
- Create exam
- Assign exam
- View results
- View analytics

Student

- Dashboard
- Assigned exams
- CBT

- Submit exam
- Results
- Performance history

System

- Auto-grading
- Basic analytics
- Exam timer
- Autosave
- Audit logging
- Tenant isolation

4. EXPLICITLY OUT OF MVP

Do not build these yet:

- AI Tutor
- AI chatbot
- Parent application
- Payroll
- Accounting
- School fees
- Transport
- Hostel management
- Biometric attendance
- Social network
- Video courses
- Marketplace
- Native mobile apps
- Adaptive learning
- Advanced predictive AI
- Complex payment infrastructure

These may become future products/features.

5. USER TYPES

5.1 Super Admin

PrepGrade internal administrator.

Can:

- manage institutions;
- suspend accounts;
- view platform metrics;
- manage platform configuration;
- review system activity.

5.2 Institution Admin

School/tutorial-centre administrator.

Can:

- manage institution;
- add teachers;
- add students;
- create classes;
- view institutional analytics;
- manage examinations.

5.3 Teacher

Can:

- manage questions;
- create exams;
- assign exams;
- view class results;
- analyse student performance.

5.4 Student

Can:

- view assigned exams;
- start exams;
- answer questions;
- submit exams;
- view own results;
- view own performance.

6. PERMISSION MATRIX

Action	Super Admin	Institution Admin	Teacher	Student
Manage platform	✓	—	—	—
Manage institution	✓	✓	—	—
Manage teachers	✓	✓	—	—
Manage students	✓	✓	Limited	—
Create questions	✓	✓	✓	—
Create exams	✓	✓	✓	—
Assign exams	✓	✓	✓	—
Take exams	—	—	Optional	✓
View own result	—	—	—	✓
View class results	✓	✓	✓	—
View institution analytics	✓	✓	—	—
View another institution	✓	—	—	—

7. INFORMATION ARCHITECTURE

PREPGRADE

```
│
├── Marketing Website
│
├── Authentication
│
├── Student
│   ├── Dashboard
```

```
│   ├── Exams
│   ├── Results
│   └── Performance
│
├── Teacher
│   ├── Dashboard
│   ├── Question Bank
│   ├── Exams
│   ├── Classes
│   ├── Results
│   └── Analytics
│
├── Institution
│   ├── Dashboard
│   ├── Students
│   ├── Teachers
│   ├── Classes
│   ├── Exams
│   └── Analytics
│
└── Super Admin
    ├── Dashboard
    ├── Institutions
    ├── Users
    ├── Exams
    └── Audit Logs
```

8. LANDING PAGE

Hero

Headline

Turn Every Assessment Into Insight.

Supporting text

Create CBT examinations, automatically grade students and discover exactly where they need to improve.

Primary CTA

Start Free

Secondary CTA

Request Demo

Sections

Problem

Traditional assessment is:

- slow;
- fragmented;
- difficult to analyse.

Solution

PrepGrade:

Create → Test → Grade → Analyse → Improve

Features

- CBT
- Question Bank
- Exam Builder
- Automatic Grading
- Student Analytics
- Teacher Analytics
- Institution Analytics

Social proof

Later:

- schools;
- student count;
- exams conducted;

- testimonials.

Do not fabricate these before they exist.

9. AUTHENTICATION FLOW

Institution registration

Register
↓
Name
Email
Phone
Password
Institution name
Institution type
↓
Verify
↓
Create institution
↓
Institution dashboard

Student invitation

Teacher/admin creates student.

System generates:

Student account / invite

Student follows invitation and creates credentials.

10. STUDENT DASHBOARD

The dashboard should immediately answer:

What do I need to do?

Header

"Good morning, Ahmed."

Cards

Assigned Exams

3

Completed

18

Average Score

72%

Current Streak

5

Upcoming Exams

Exam	Subject	Date	Duration	Action
Mathematics Mock	Mathematics	Today	60 min	Start
Biology Test	Biology	Tomorrow	45 min	View

Performance

Overall

72%

Mathematics

65%

English

81%

Physics

59%

Weak Area

Physics — Electricity

Accuracy: 43%

CTA:

Practise Topic

This button can initially lead to a simple filtered practice experience later. It does not need to be part of MVP if practice mode isn't ready.

11. TEACHER DASHBOARD

Teacher's first screen:

Overview

Students: 126

Exams: 18

Average score: 69%

Assessments this week: 5

Recent Exams

Mathematics Test
SS2A
42 students
Average: 71%

Weak Topics

Probability 44%
Trigonometry 52%
Algebra 58%

CTA:

Create Revision Test

12. QUESTION BANK

Main screen

Filters:

- Subject
- Topic
- Difficulty
- Question type
- Status

Search:

Search questions...

Actions:

Create Question

Question card

Mathematics
Algebra
Medium

Solve:
2x + 5 = 15

A. 3
B. 5
C. 7
D. 10

Correct: B

Status: Approved

13. CREATE QUESTION

Fields:

Subject

Required.

Topic

Required.

Question

Required.

Question type

MVP:

Multiple Choice

Future:

- True/False
- Multiple response
- Short answer
- Essay

Options

A

B

C

D

Correct answer

Required.

Difficulty

- Easy
- Medium
- Hard

Explanation

Optional for MVP.

Source

Optional but strongly recommended.

Copyright/licensing metadata

Important for commercial content.

14. EXAM BUILDER

Teacher clicks:

Create Exam

First:

Basic Information

Exam name:

SS2 Mathematics Mid-Term Test

Subject:

Mathematics

Class:

SS2A

Duration:

60 minutes

Questions

Teacher can:

Add from Question Bank

or:

Create New Question

Exam Settings

- Duration
- Number of attempts
- Randomise questions
- Randomise options
- Show result immediately
- Allow review
- Start date
- End date

15. EXAM STATES

DRAFT
↓
PUBLISHED
↓

SCHEDULED
↓
ACTIVE
↓
COMPLETED
↓
ARCHIVED

A published exam should not be freely edited once students have started attempts.

If critical changes are required:

Create a new version.

This protects result integrity.

16. STUDENT EXAM FLOW

Assigned Exam
↓
Instructions
↓
Confirm Start
↓
CBT
↓
Submit
↓
Processing
↓
Result

17. CBT INTERFACE

Top bar

Mathematics Mock Examination

Question 14 / 50

Time Remaining
00:42:18

Question

If 2x + 5 = 15, what is x?

Options:

○ 3

○ 5

○ 7

○ 10

Bottom

Previous

Mark for Review

Next

Question navigator

1 2 3 4 5
6 7 8 9 10
...

18. CBT FAILURE HANDLING

This is a major acceptance criterion.

Scenario

Student answers question 10.

Internet disappears.

Expected:

1. Answer remains locally available.
2. System attempts synchronization.
3. Connection returns.
4. Answer synchronizes.
5. Student continues.

The student should not be forced to restart the examination.

19. RESULT SCREEN

Immediately after submission:

76%

Performance

Correct: 38

Wrong: 9

Unanswered: 3

Time

47m 21s

Topic performance

Topic	Accuracy
Algebra	82%
Geometry	74%
Probability	43%

Recommendation

Probability is currently your weakest area.

This recommendation can initially be rule-based rather than AI-generated.

20. TEACHER RESULT SCREEN

Teacher can see:

Mathematics Mock Examination

Students: 42
Completed: 40
Average: 68%

Highest: 94%
Lowest: 31%

Table:

Student	Score	%	Status
Student A	47/50	94%	Excellent
Student B	41/50	82%	Good
Student C	22/50	44%	Needs Support

21. INSTITUTION ANALYTICS

Institution dashboard:

Overall

Students: 642

Teachers: 38

Exams: 214

Average score: 68%

Subject performance

English 76%
Biology 73%
Chemistry 69%
Mathematics 61%
Physics 58%

At-risk students

For MVP, define a transparent rule such as:

Students below a configurable threshold.

Example:

Below 40% = Needs Attention

Do not call this an AI prediction yet.

22. DATABASE SCHEMA

A stronger MVP schema:

users
---
id
email
phone
full_name
avatar_url
role
created_at
updated_at
institutions
---
id
name
slug

logo_url
type
status
owner_id
created_at
updated_at
institution_members
---
id
institution_id
user_id
role
status
created_at
students
---
id
institution_id
user_id
student_code
class_id
created_at
teachers
---
id
institution_id
user_id
created_at
classes
---
id
institution_id
name
academic_session
created_at
subjects
---
id
name
code
topics
---
id

subject_id
name
questions
---
id
institution_id
subject_id
topic_id
created_by
question_text
question_type
difficulty
explanation
source
status
created_at
updated_at
question_options
---
id
question_id
option_text
option_key
is_correct
exams
---
id
institution_id
created_by
title
description
subject_id
duration_minutes
attempts_allowed
randomize_questions
randomize_options
start_at
end_at
status
created_at
updated_at
exam_questions
---

id
exam_id
question_id
position
marks
exam_assignments
---
id
exam_id
class_id
student_id
assigned_at
exam_attempts
---
id
exam_id
student_id
started_at
submitted_at
status
score
percentage
time_used
attempt_answers
---
id
attempt_id
question_id
selected_option_id
is_correct
answered_at
performance_metrics
---
id
student_id
subject_id
topic_id
attempt_id
accuracy
questions_attempted
created_at
audit_logs
---

id
institution_id
user_id
action
entity_type
entity_id
metadata
created_at

23. API DESIGN

If using REST initially:

Authentication

POST /auth/register
POST /auth/login
POST /auth/logout
POST /auth/reset-password

Institutions

GET /institutions/:id
PATCH /institutions/:id
GET /institutions/:id/students
GET /institutions/:id/teachers
GET /institutions/:id/classes

Questions

GET /questions
POST /questions
GET /questions/:id
PATCH /questions/:id
DELETE /questions/:id

Exams

GET /exams
POST /exams

GET /exams/:id
PATCH /exams/:id
POST /exams/:id/publish
POST /exams/:id/assign

Attempts

POST /exams/:id/start
POST /attempts/:id/answers
POST /attempts/:id/sync
POST /attempts/:id/submit
GET /attempts/:id

Results

GET /results/:attemptId
GET /students/:studentId/results
GET /classes/:classId/results
GET /institutions/:institutionId/analytics

24. API SECURITY

Every protected API request should verify:

Authentication
↓
User identity
↓
Institution membership
↓
Role
↓
Resource ownership
↓
Permission

Never rely solely on frontend route protection.

For example, hiding:

/admin

from a teacher is **not security**.

The backend/database must also reject unauthorized access.

25. NOTIFICATION SYSTEM

MVP can keep this simple.

Potential notifications:

Student

Mathematics Mock Examination has been assigned to you.

Teacher

38 students completed your Mathematics examination.

Institution

Weekly assessment report is ready.

Start with in-app notifications.

Email/WhatsApp/SMS can come later.

26. IMPORT SYSTEM

One high-value feature I would add relatively early:

CSV student import

A school may have 500 students.

Nobody wants to manually enter 500 students.

Allow:

Upload CSV

Example:

First Name
Last Name
Student ID
Email
Class

Then:

Validate → Preview → Import

This can significantly improve onboarding.

27. QUESTION IMPORT

Similarly, allow teachers to upload structured question files eventually.

For example:

Subject
Topic
Question
Option A
Option B
Option C
Option D
Correct Answer
Difficulty

Then:

Upload → Validate → Preview → Import

This can dramatically accelerate question-bank creation.

28. REPORTING

MVP should provide downloadable:

Student report

- exam;
- score;
- percentage;
- correct;
- wrong;
- unanswered;
- topic performance.

Class report

- student scores;
- average;
- highest;
- lowest;
- topic performance.

Institution report

- overall performance;
- subject performance;
- class performance.

PDF/Excel exports can be introduced after basic reporting works.

29. NON-FUNCTIONAL REQUIREMENTS

Performance

Common dashboard pages should load quickly under normal network conditions.

Reliability

Exam submission must be highly reliable.

Scalability

Architecture should support increasing institutions without redesigning the entire platform.

Accessibility

- readable typography;
- keyboard navigation where practical;
- sufficient contrast;
- clear error messages.

Responsiveness

Support:

- mobile;
- tablet;
- desktop.

30. SECURITY REQUIREMENTS

Minimum:

- HTTPS;
- secure authentication;
- password hashing through trusted auth provider;
- RBAC;
- database row-level security;
- tenant isolation;
- audit logs;
- input validation;
- rate limiting;
- secure API authorization;
- backups;
- secret management;
- dependency updates.

For Nigeria, privacy compliance should be incorporated into the commercial architecture from the beginning rather than treated as a post-launch task.

31. OBSERVABILITY

The engineering team should know when something breaks.

Track:

- API errors;
- failed exam submissions;
- authentication failures;
- database errors;
- slow requests;
- failed background jobs;
- uptime;
- unusual traffic.

Critical event:

Exam submission failure

should trigger high-priority monitoring.

32. ANALYTICS EVENTS

Instrument the product from day one.

Events:

user_registered
institution_created
teacher_added
student_added
question_created
question_published
exam_created
exam_published
exam_started
question_answered
exam_submitted
result_viewed

teacher_viewed_analytics
student_viewed_performance

Then you can answer:

Where are users dropping out?

33. PRODUCT FUNNEL

Measure:

Registered
↓
Institution Created
↓
Teacher Added
↓
First Question
↓
First Exam
↓
First Student Attempt
↓
First Result
↓
Second Exam
↓
30-Day Retention
↓
Paid Subscription

The most important early milestone:

First successful assessment

34. DEFINITION OF DONE

A feature isn't finished because the developer says:

"It works."

It is finished when:

- functionality works;
- mobile layout works;
- permissions work;
- validation works;
- errors are handled;
- database constraints work;
- security is tested;
- analytics event exists where appropriate;
- acceptance criteria pass.

35. EXAMPLE DEVELOPER TICKET

PG-001 — Create Question

Priority: P0

User story

As a teacher, I want to create a multiple-choice question so that I can use it in an examination.

Acceptance criteria

- Teacher can select subject.
- Teacher can select topic.
- Teacher can enter question.
- Teacher can enter four options.
- Teacher must select exactly one correct answer.
- Teacher can select difficulty.
- Required fields are validated.
- Question is saved to database.
- Question belongs to teacher's institution.

- Another institution cannot access it.

Status

TODO

36. SAMPLE MVP BACKLOG

EPIC 1 — Authentication

- PG-001 Registration
- PG-002 Login
- PG-003 Logout
- PG-004 Password reset
- PG-005 Role management

EPIC 2 — Institutions

- PG-006 Create institution
- PG-007 Institution settings
- PG-008 Teacher management
- PG-009 Student management
- PG-010 Class management

EPIC 3 — Questions

- PG-011 Create question
- PG-012 Edit question
- PG-013 Archive question
- PG-014 Search questions
- PG-015 Filter questions

EPIC 4 — Exams

- PG-016 Create exam
- PG-017 Add questions
- PG-018 Exam settings
- PG-019 Publish exam

- PG-020 Assign exam

EPIC 5 — CBT

- PG-021 Exam instructions
- PG-022 Start attempt
- PG-023 Question navigation
- PG-024 Answer question
- PG-025 Mark for review
- PG-026 Autosave
- PG-027 Submit exam

EPIC 6 — Results

- PG-028 Grade attempt
- PG-029 Result screen
- PG-030 Exam history
- PG-031 Student analytics
- PG-032 Class analytics

EPIC 7 — Security

- PG-033 RLS
- PG-034 RBAC
- PG-035 Audit logging
- PG-036 Rate limiting
- PG-037 Security testing

EPIC 8 — Pilot

- PG-038 Onboarding
- PG-039 CSV student import
- PG-040 Feedback system
- PG-041 Usage dashboard
- PG-042 Pilot reporting

37. PRODUCT ROADMAP AFTER MVP

Once the core product is validated:

Phase 2

Commercial SaaS

- subscriptions;
- billing;
- institution branding;
- advanced reports;
- CSV imports;
- exports;
- notifications.

Phase 3

Assessment Intelligence

- weakness detection;
- mastery scores;
- longitudinal analytics;
- intervention recommendations.

Phase 4

AI

- AI question generator;
- AI question explanation;
- AI assessment creation;
- AI teacher assistant.

Phase 5

Adaptive Learning

The system begins dynamically selecting questions based on performance.

Example:

Student answers five easy questions correctly.

System increases difficulty.

Student struggles.

System reduces difficulty and identifies the concept requiring remediation.

38. AI QUESTION GENERATOR — FUTURE DESIGN

Teacher selects:

Subject: Mathematics
Topic: Probability
Class: SS2
Difficulty: Medium
Questions: 10

AI generates drafts.

But the workflow is:

AI
↓
Draft Questions
↓
Teacher Review
↓
Edit
↓
Approve

↓
Question Bank

Never:

AI → Automatically publish

39. MASTER DATA ADVANTAGE

Over time, PrepGrade could accumulate a structured dataset containing:

- question difficulty;
- topic;
- student attempts;
- accuracy;
- time spent;
- common wrong answers;
- improvement;
- assessment history.

That could eventually power:

PrepGrade Assessment Intelligence Engine

This is potentially much more defensible than simply having a nice CBT interface.

40. BUSINESS MODEL EVOLUTION

Stage 1

Free pilots.

↓

Stage 2

Institution subscriptions.

↓

Stage 3

Premium analytics.

↓

Stage 4

Enterprise licensing.

↓

Stage 5

Assessment API.

↓

Stage 6

African assessment infrastructure.

41. THE FIRST REVENUE MODEL

I would initially keep pricing extremely simple.

For example:

Starter

Small institutions.

Growth

Medium institutions.

Enterprise

Large institutions.

Do not launch ten pricing packages.

The sales conversation should be:

How many students do you need to assess?

rather than:

"Which of our 17 complicated packages would you like?"

42. PILOT CONTRACT

Every serious pilot should have a simple written agreement covering:

- pilot duration;
- number of users;
- responsibilities;
- support;
- data handling;
- content ownership;
- termination;
- pricing after pilot;
- confidentiality where appropriate.

Have Nigerian legal counsel review the commercial/privacy terms before broad deployment.

43. THE FIRST 90 DAYS OF THE COMPANY

DAYS 1–30

Validate

- customer interviews;
- competitor analysis;
- prototype;
- pilot recruitment.

Goal

3–5 committed pilot institutions.

DAYS 31–60

Build

- authentication;
- institution;
- teachers;
- students;
- question bank;
- exams;
- CBT.

Goal

Working MVP.

DAYS 61–90

Deploy

- real exams;
- real students;
- real teachers;
- real analytics.

Goal

Evidence of recurring usage.

44. GO / NO-GO FRAMEWORK

After the pilot, score PrepGrade.

GO

If:

- users complete exams;
- teachers repeatedly create assessments;
- institutions return;
- customers pay;
- major bugs are manageable.

ITERATE

If:

- users like the concept;
- usage is inconsistent;
- onboarding is difficult;
- product needs refinement.

PIVOT

If:

- institutions don't care;
- another problem is much more valuable;
- willingness to pay is weak.

STOP

If:

- repeated validation shows no meaningful demand.

Stopping a bad direction early is not failure.

It is capital preservation.

45. THE MOST IMPORTANT FINANCIAL RULE

Never make this mistake:

Build a ₦10m product and then ask whether anyone wants it.

Instead:

₦0
↓
Research
↓
Prototype
↓
Small MVP
↓
Pilot
↓
First revenue
↓
Retention
↓
Reinvestment
↓
Scale

Capital should increase **because evidence increases**.

46. YOUR ROLE AS FOUNDER

For PrepGrade, I would position you initially as:

Founder & Product/Technology Lead

Your responsibilities:

Product

Understand the problem.

Technology

Understand architecture and development.

Sales

Get the first institutions.

Customer Success

Make pilot customers successful.

Strategy

Decide what NOT to build.

Your biggest job isn't writing every line of code.

It is:

Making sure the company builds the right thing.

47. FIRST 10 CUSTOMERS

I would personally treat the first ten customers as a research laboratory.

For each one, document:

Institution
Number of students
Number of teachers
Exams/month
Questions/month
Most-used feature
Least-used feature
Problems
Retention

Payment
Feature requests

Then compare them.

Patterns will emerge.

Those patterns should determine the next version of PrepGrade.

48. THE FIRST MAJOR PRODUCT MILESTONE

Don't call it:

Version 1.0.

Call it:

PrepGrade Pilot 1

Success means:

A real institution can conduct a real examination from beginning to end without your intervention.

That means:

Teacher creates exam → students take exam → system grades → teacher analyses results.

When that works reliably:

you have a real product.

49. THE SECOND MILESTONE

PrepGrade Paid 1

Success means:

A real institution voluntarily pays to continue using the system.

This is more important than thousands of free registrations.

50. THE THIRD MILESTONE

PrepGrade Retention 1

Success means:

Institutions continue using PrepGrade for multiple assessment cycles.

This demonstrates that the product is becoming part of their workflow.

51. THE FOURTH MILESTONE

PrepGrade Intelligence 1

Success means:

PrepGrade can identify student weaknesses and help teachers act on them.

Now you have something substantially more differentiated.

52. THE FIFTH MILESTONE

PrepGrade Scale 1

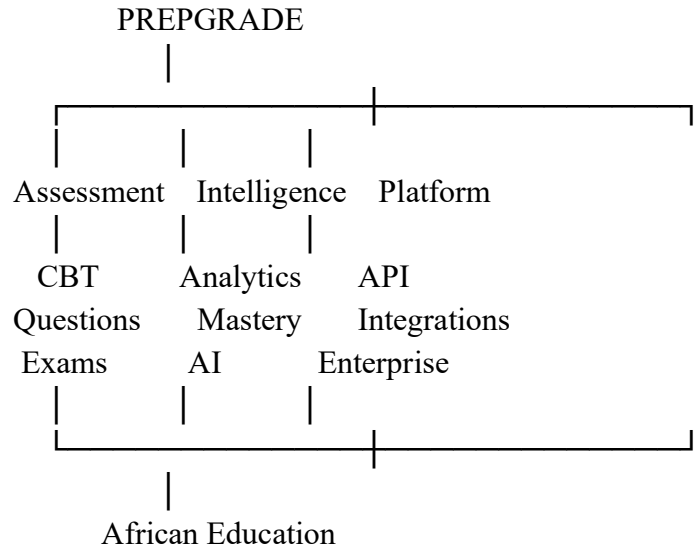
Success means:

The company can acquire and onboard institutions repeatedly without the founder manually doing everything.

That is when you start building the organisation.

53. THE FINAL COMPANY ARCHITECTURE

Long term, I would structure the company into:



The initial product is small.

The architecture is ambitious.

That distinction is important.

54. THE STRATEGIC ENDGAME

I would not build PrepGrade merely to become:

"another CBT app."

The endgame is:

Assessment Intelligence Infrastructure for Africa

Imagine a future where:

A school uses PrepGrade.

A teacher creates an assessment.

500 students take it.

PrepGrade grades it.

The system discovers:

Mathematics → Algebra → Weak

The teacher receives:

"42% of SS2 students are struggling with quadratic equations."

The system creates a targeted assessment.

Students practise.

A week later:

Algebra performance increased from 51% → 67%.

Now PrepGrade is no longer merely conducting exams.

It is helping the institution **improve learning outcomes**.

That is the long-term company.

55. IMMEDIATE ACTION PLAN

Your next execution sequence should be:

WEEK 1

Product discovery

- Interview 10 teachers.
- Interview 5 school administrators.
- Interview 10 students.
- Interview 5 tutorial-centre operators.

WEEK 2

Product design

- Finalise MVP.
- Create wireframes.
- Create database architecture.
- Create technical architecture.

WEEK 3

Prototype

- Build clickable prototype.
- Demonstrate to potential pilot customers.
- Collect objections.

WEEK 4

Pilot commitment

Secure:

3–5 pilot institutions.

WEEKS 5–12

Build → Test → Deploy → Measure → Improve.

56. THE ONE SENTENCE I WANT THE ENTIRE TEAM TO REMEMBER

Do not build what sounds impressive; build what makes a teacher, student, or institution say: "I need this."

And the development sequence remains:

MVP

↓

PILOT SCHOOLS

↓

PAYING CUSTOMERS

↓

RETENTION

↓

ASSESSMENT INTELLIGENCE

↓

AI

↓

NATIONAL SCALE

↓

AFRICAN EXPANSION

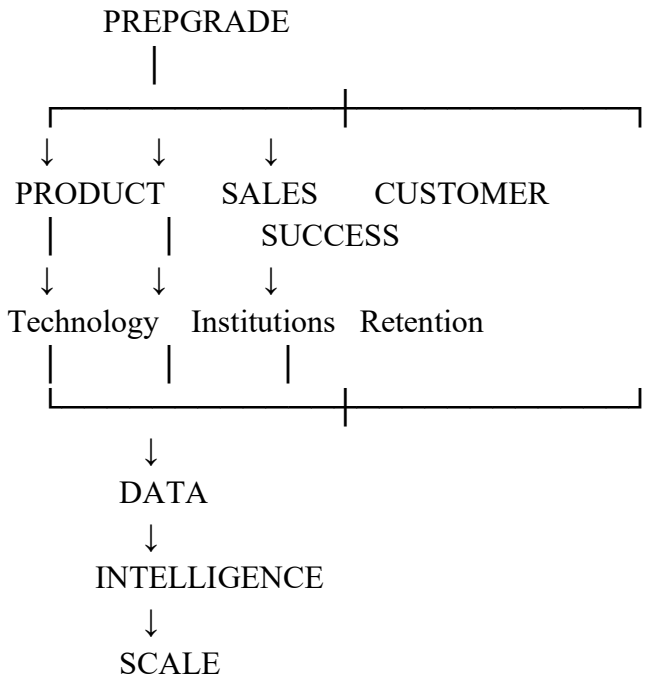
That is how I would build PrepGrade without burning millions before product-market fit.

PREPGRADE — PHASE 4

Go-to-Market, Operations, Project Management & Scale Plan

1. THE PREPGRADE OPERATING MODEL

PrepGrade should operate around five engines:



The mistake many startups make is building only the **Product** engine.

A real company needs all five.

2. COMPANY DEPARTMENTS

You do not need all of these employees on day one.

But the company should eventually have:

Executive

- CEO
- CTO
- COO
- CFO/Finance

Product

- Product Manager
- UI/UX Designer
- Product Researcher

Engineering

- Frontend Engineers
- Backend Engineers
- Mobile Engineer
- DevOps/Cloud Engineer
- QA Engineer
- Security Engineer

Education

- Academic Lead
- Subject Specialists
- Content Reviewers

Growth

- Sales
- Marketing
- Partnerships
- Customer Success

Operations

- Customer Support
- Finance
- Legal
- HR

3. INITIAL TEAM

Do **not** hire the whole structure.

The initial team can be:

Founder

CEO + Product

Technical Lead

Architecture + development

Full-stack developer

Product development

UI/UX designer

Part-time/freelance initially

Academic advisor

Part-time

Sales/customer success

Founder initially

That's enough to begin.

4. PROJECT MANAGEMENT STRUCTURE

Use a simple hierarchy:

VISION
↓
OBJECTIVES
↓
EPICS
↓
FEATURES
↓
USER STORIES
↓
TASKS
↓
TESTING

↓
RELEASE

For example:

Objective

Launch reliable CBT MVP.

↓

Epic

Examination Engine.

↓

Feature

Student can start examination.

↓

User Story

As a student, I want to start my assigned exam.

↓

Tasks

- Build start endpoint.
- Build start screen.
- Create attempt record.
- Start timer.
- Save start timestamp.
- Test.

5. PROJECT MANAGEMENT TOOL

Use one project-management system.

Do not manage engineering through WhatsApp messages.

Recommended structure:

Backlog

Everything that may eventually be built.

Sprint

Work selected for the current period.

In Progress

Currently being developed.

Review

Awaiting code/product review.

QA

Being tested.

Done

Meets Definition of Done.

6. TWO-WEEK SPRINTS

I recommend:

2-week development sprints.

Each sprint:

Day 1

Planning.

Days 2–8

Development.

Days 9–10

Testing and bug fixing.

Day 10

Demo + retrospective.

Then repeat.

7. SPRINT PLANNING

Before a sprint begins, ask:

What is the most important outcome?

Not:

"How many tickets can we finish?"

Instead:

"What user capability will exist at the end?"

Example:

Sprint 1 objective

Institution administrators can create an institution and manage teachers/students.

Everything that doesn't contribute to that outcome is postponed.

8. DAILY ENGINEERING ROUTINE

Every developer should answer three questions:

Yesterday

What did I complete?

Today

What am I working on?

Blocker

What is preventing progress?

Keep it short.

9. PRODUCT DECISION PROCESS

Every feature request goes through:

Request
↓
Problem
↓
Evidence
↓
Impact
↓
Effort
↓
Priority
↓
Decision

Never build something simply because:

"A customer asked."

Ask:

"Is this a problem shared by enough customers?"

10. PRIORITY FRAMEWORK

Use:

Impact × Confidence ÷ Effort

Example:

CSV student import

Impact: High

Confidence: High

Effort: Medium

→ Build.

Animated student avatars

Impact: Low

Confidence: Low

Effort: Medium

→ Don't build.

11. PREPGRADE DESIGN SYSTEM

The product should feel:

- modern;
- professional;
- trustworthy;
- fast;
- clean.

Think:

Stripe + Linear + Notion + modern education software

rather than a cluttered traditional school portal.

12. DESIGN PRINCIPLES

White space

Do not cram everything onto dashboards.

Strong hierarchy

Users should immediately know:

What do I do next?

Data visualization

Use charts only when they communicate something useful.

Consistency

Buttons, forms, tables and navigation should behave consistently.

Mobile-first

A teacher should be able to manage an exam from a phone.

13. STUDENT EXPERIENCE

Students should not feel like they're using administrative software.

The student interface should feel like a modern learning application.

Home

"Good evening, Umar."

Then:

Your next exam

Mathematics Mock

Today — 7:00 PM

Start Exam

Simple.

14. TEACHER EXPERIENCE

Teachers should feel:

"This saves me time."

The teacher shouldn't need five pages to create an exam.

Ideal workflow:

Create Exam
↓
Choose Subject
↓
Select Questions
↓
Choose Class
↓
Set Time
↓
Publish

15. INSTITUTION EXPERIENCE

Administrators need:

Control

Who has access?

Visibility

How are students performing?

Reliability

Are exams working?

Reports

What happened?

Management

Which teachers/classes are active?

16. THE "ZERO TRAINING" TEST

A new teacher should be able to:

Create and publish an examination in under 10 minutes without speaking to PrepGrade support.

If they cannot:

the product needs simplification.

17. PILOT SALES STRATEGY

The first sales team is:

You.

Do not immediately hire salespeople.

You need to learn:

- what customers say;
- what objections they have;
- why they buy;
- why they reject;
- what they value.

Once you understand that, train salespeople.

18. YOUR FIRST CUSTOMER TARGET

Start with:

Private secondary schools

and

JAMB/tutorial centres.

Why?

They have:

- recurring examinations;
- decision-makers;
- immediate assessment needs;
- measurable outcomes.

19. PROSPECT LIST

Build a spreadsheet:

Institution	Contact	Decision Maker	Students	Current System	Status
School A	...	Principal	450	Manual	Contacted
Centre B	...	Director	800	WhatsApp	Demo
School C	...	ICT Manager	600	Software	Pilot

Statuses:

id="1n5n8n"
PROSPECT
CONTACTED
RESPONDED
DEMO
PILOT
ACTIVE

PAID
LOST

20. SALES SCRIPT

Your opening should not be:

"Hello, I built an EdTech platform."

Instead:

"We're testing a new assessment platform that helps schools conduct CBT examinations, automatically grade students and identify weak topics. I'd like to understand how your school currently handles examinations and see whether PrepGrade could solve some of the problems."

Then listen.

21. DISCOVERY QUESTIONS

Ask:

1. How do you currently conduct examinations?
2. How many students take exams?
3. How much time does grading take?
4. Do you currently use CBT?
5. What software do you use?
6. What do teachers dislike about it?
7. How do you analyse weak topics?
8. Do parents receive reports?
9. What would make you change systems?
10. Would you pay for a better solution?

22. DEMO STRATEGY

Never give a 60-minute presentation.

Give a:

10-minute demonstration.

Minute 1

Problem.

Minutes 2–4

Teacher creates exam.

Minutes 5–7

Student takes exam.

Minutes 8–9

Automatic result.

Minute 10

Analytics.

Then ask:

"Would this solve a real problem for your institution?"

23. PILOT OFFER

Create:

PrepGrade Founding Institution Programme

Benefits:

- early access;
- discounted pricing;
- direct founder support;

- influence over product roadmap;
- founding institution status.

But make the offer time-limited.

For example:

"We're selecting the first five institutions."

Scarcity should be genuine, not manufactured.

24. PILOT SUCCESS AGREEMENT

Before onboarding, establish:

Institution agrees to:

- provide feedback;
- use the system regularly;
- conduct real assessments;
- nominate a product champion;
- participate in review meetings.

PrepGrade agrees to:

- onboarding;
- technical support;
- bug fixing;
- product improvements;
- data protection practices.

25. CUSTOMER SUCCESS

Every institution gets a:

Customer Success Score

Example:

Login activity 90%
Teacher activity 80%
Exam activity 95%
Student activity 87%
Support issues Low
Renewal likelihood High

Then classify:

Green

Healthy.

Yellow

Needs attention.

Red

At risk.

26. RETENTION SYSTEM

If an institution hasn't created an exam for 14 days:

Don't immediately send spam.

Investigate:

"Is there anything preventing your teachers from using PrepGrade?"

Possible causes:

- school holiday;
- technical problem;
- teacher confusion;
- insufficient content;
- lack of perceived value.

Then solve the real issue.

27. PRODUCT RETENTION LOOP

PrepGrade should eventually become:

Weekly

Teachers conduct tests.

Monthly

School reviews analytics.

Termly

School generates performance reports.

Annually

School renews subscription.

That creates recurring value.

28. THE "AHA MOMENT"

For PrepGrade, the ideal Aha Moment is:

A teacher finishes an exam and immediately sees exactly which topics the class is struggling with.

That is the moment where PrepGrade becomes more than CBT.

Everything should move toward making this experience excellent.

29. MONETIZATION MILESTONES

Don't obsess over revenue before product validation.

Stage 1

Free pilot.

Stage 2

Paid pilot.

Stage 3

Annual subscriptions.

Stage 4

Expansion revenue.

Stage 5

Enterprise contracts.

30. UNIT ECONOMICS

Eventually track:

CAC

Customer Acquisition Cost.

Example:

You spend ₦200,000 on sales/marketing and acquire 10 paying institutions.

CAC:

₦20,000

ARPA

Average Revenue Per Account.

LTV

Lifetime Value.

Churn

Percentage of customers leaving.

Gross Margin

Revenue minus direct infrastructure/service costs.

A healthy SaaS company should eventually have:

LTV significantly greater than CAC.

Don't optimize these numbers before you have enough customers to measure them reliably.

31. REVENUE EXAMPLE

Suppose eventually:

100 schools

pay an average:

₦500,000/year.

That gives:

₦50 million annual recurring revenue.

At:

1,000 schools:

₦500 million/year.

At:

10,000 schools:

₦5 billion/year.

These are illustrative scenarios, **not forecasts**.

The important point is:

Recurring institutional revenue can become significant without charging every student individually.

32. EXPANSION REVENUE

Suppose a school begins with:

Basic CBT

Then adds:

Advanced Analytics

Then:

AI Assessment

Then:

Additional student capacity

Then:

White-label branding

This creates:

Net Revenue Expansion

instead of constantly finding new customers.

33. CUSTOMER EXPANSION

A 300-student school may eventually become:

500 students.

Then:

800.

Then:

1,500.

PrepGrade's revenue can increase without acquiring an entirely new institution.

34. VIRALITY

PrepGrade can eventually benefit from natural distribution.

Teacher creates exam.

Students receive:

"Take your exam on PrepGrade."

Students discover the platform.

Students tell other teachers.

Teachers tell other schools.

But do not rely on virality.

Build sales and partnerships first.

35. PARTNERSHIP STRATEGY

After product-market fit, approach:

- tutorial networks;
- education associations;
- teacher communities;
- education NGOs;
- publishers;

- training organisations;
- school networks;
- education technology companies.

Long term:

- universities;
- government programmes;
- large education initiatives.

36. ENTERPRISE STRATEGY

Large organisations may need:

- dedicated support;
- custom branding;
- custom reporting;
- integrations;
- SLAs;
- large-scale user management;
- dedicated environments.

Do not build enterprise complexity until enterprise demand exists.

37. API BUSINESS

Eventually:

PrepGrade Assessment API

Other companies could integrate PrepGrade.

Example:

A learning platform wants exams.

Instead of building:

- question bank;
- grading;

- analytics;
- attempts;

it integrates PrepGrade.

This creates a second business model.

38. WHITE-LABEL

Eventually:

Powered by PrepGrade

could become:

Powered by [Institution Brand]

Large educational organisations could have their own branded assessment environment.

This can become a high-value enterprise product.

39. DATA ASSET — BUT WITH ETHICS

Assessment data could become extremely valuable.

But never treat students' personal data as something to casually sell.

PrepGrade should build its data advantage through:

- legitimate collection;
- transparency;
- consent/legal basis where required;
- minimisation;
- anonymisation/pseudonymisation where appropriate;
- strong access control.

The commercial moat should be:

better intelligence

not:

selling children's data.

40. AI STRATEGY

The order should be:

Phase 1

Rules.

Phase 2

Analytics.

Phase 3

Machine learning.

Phase 4

LLM features.

Phase 5

Adaptive intelligence.

This prevents the company from becoming:

"An AI wrapper with no real data infrastructure."

41. FIRST AI FEATURE

I would choose:

AI Assessment Generator

because it directly benefits the paying customer:

teacher.

Teacher:

SS3 Physics — Waves — 20 medium questions.

AI:

Generates draft.

Teacher:

Reviews.

Teacher:

Publishes.

This saves time.

That's measurable ROI.

42. SECOND AI FEATURE

AI Performance Explanation

Teacher sees:

Class average in Probability: 47%.

PrepGrade:

"Most incorrect responses were concentrated around conditional probability and independent events."

Then:

Create Revision Assessment

This connects AI directly to the assessment loop.

43. THIRD AI FEATURE

Student AI Study Assistant

Only after the assessment engine is strong.

Student:

"Why am I weak in probability?"

PrepGrade responds based on the student's actual performance and approved learning resources.

Not generic hallucinated tutoring.

44. LONG-TERM PERSONALISATION

Eventually:

Student
↓
Knowledge Profile
↓
Weak Concepts
↓
Recommended Questions
↓
Answers
↓
Updated Knowledge Profile

This creates:

Adaptive Assessment.

45. PREPGRADE CONTENT STRATEGY

You need three content layers.

Layer 1

Institution-created content.

Layer 2

PrepGrade-owned/licensed content.

Layer 3

AI-generated draft content reviewed by humans.

Never build the business around unlicensed copied past questions.

46. QUESTION QUALITY SCORE

Eventually every question can receive:

Difficulty
Discrimination
Accuracy
Average Time
Error Rate
Ambiguity Reports
Topic
Learning Objective

If 90% of strong students answer a question correctly but weak students fail:

That question may have good discrimination.

Over time, PrepGrade can develop sophisticated assessment science.

47. LONG-TERM ASSESSMENT ENGINE

Eventually the engine could dynamically choose questions.

Example:

Student begins at:

Medium

Answers correctly:

→ harder.

Answers incorrectly:

→ similar question.

Repeated failure:

→ foundational question.

Then:

Mastery confirmed.

This is much more powerful than static CBT.

48. AFRICAN EXPANSION

Do not expand to ten countries simultaneously.

Use:

Nigeria
↓
One additional market
↓
Validate

↓
Improve localisation
↓
Second market
↓
Regional expansion

Potential future markets should be evaluated individually based on:

- examination structure;
- digital adoption;
- regulatory environment;
- competition;
- school software spending;
- language;
- distribution.

49. 5-YEAR COMPANY EVOLUTION

YEAR 1

Product

CBT + Assessment.

Objective

Validation.

YEAR 2

Product

Assessment SaaS.

Objective

Product-market fit.

YEAR 3

Product

Assessment Intelligence.

Objective

National growth.

YEAR 4

Product

AI + Adaptive Assessment.

Objective

Market leadership.

YEAR 5

Product

Assessment Infrastructure.

Objective

Africa.

50. FUNDING STRATEGY

I would use:

Milestone-Based Funding.

Before MVP

Minimal capital.

After MVP

Still minimal.

After pilots

Reinvest revenue.

After paying customers

Consider angel/pre-seed funding.

After strong recurring revenue

Consider institutional venture funding.

Do not raise money simply because:

"Every startup raises money."

Raise because:

Capital can accelerate something already working.

51. WHAT INVESTORS WILL EVENTUALLY WANT TO SEE

Product

Does it work?

Customers

Who uses it?

Revenue

Who pays?

Retention

Do they stay?

Growth

Is the company growing?

Market

Can this become large?

Moat

Why can't another company copy it?

Team

Can the founders execute?

Economics

Can this become profitable?

52. INVESTOR DASHBOARD

Eventually your investor dashboard should show:

MRR
ARR
Active Institutions
New Institutions
Churn
Retention
CAC
ARPA
Gross Margin

Students
Teachers
Assessments
Monthly Attempts
Average Institution Usage

But remember:

In the beginning, **usage and retention matter more than vanity metrics.**

53. FOUNDER WEEKLY DASHBOARD

You personally should review every Sunday:

Product

- bugs;
- releases;
- active users.

Sales

- prospects;
- demos;
- pilots;
- payments.

Customers

- active institutions;
- inactive institutions;
- complaints.

Finance

- revenue;
- expenses;
- runway.

Strategy

- what worked?
- what didn't?
- what should we stop?

54. WEEKLY CEO SCORECARD

Example:

Metric	Target
New prospects	20
Demos	5
Pilot conversations	3
Product releases	1
Customer interviews	5
Critical bugs	0
Founder sales hours	10+

Adjust targets as the company develops.

55. MONTHLY BOARD-STYLE REVIEW

Even before you have investors, behave like you do.

Every month create:

1. Executive summary

What happened?

2. Product

What shipped?

3. Customers

What changed?

4. Revenue

How much?

5. Retention

Who stayed/left?

6. Problems

What is broken?

7. Decisions

What are we changing?

8. Next month

Top five priorities.

This creates professional management discipline.

56. THE "STOP BUILDING" LIST

Every month create:

Features we will NOT build.

This is surprisingly important.

For example:

- parent app;
- payroll;
- transport;
- social feed;
- cryptocurrency;
- unnecessary AI chatbot.

This protects the team from scope creep.

57. SCOPE CREEP CONTROL

Any new feature request must answer:

Who needs it?

How many customers need it?

What problem does it solve?

How frequently?

Revenue impact?

Retention impact?

Development cost?

Security impact?

If nobody can answer these:

Backlog.

58. THE PREPGRADE FLYWHEEL

The long-term company should have this flywheel:

More Institutions
↓
More Assessments
↓
More Performance Data
↓
Better Analytics
↓
Better Recommendations
↓

Better Student Outcomes
↓
Higher Institutional Value
↓
Higher Retention
↓
More Institutions

That is the moat I would aim to build.

59. THE MOST IMPORTANT STRATEGIC DECISION

PrepGrade should NOT attempt to be:

"Everything a school needs."

There are already established school-management systems competing for that space.

Instead:

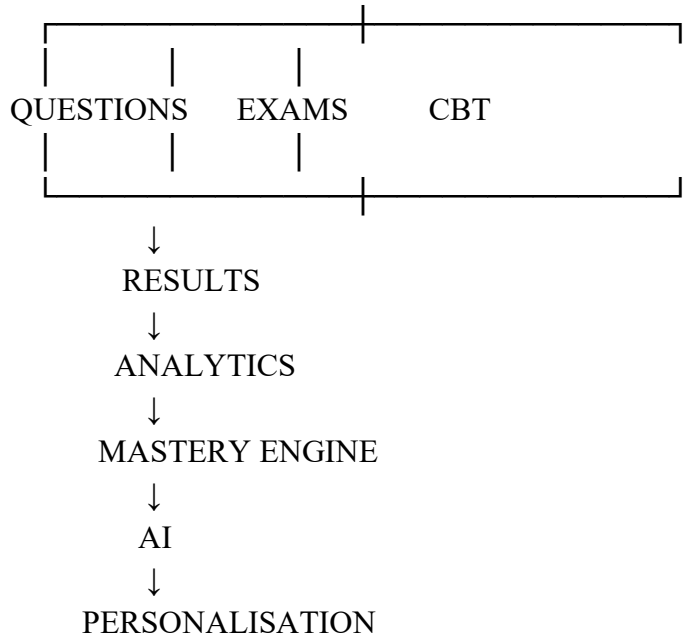
"Everything an institution needs to assess, understand and improve student performance."

That is much sharper.

60. THE FINAL PRODUCT HIERARCHY

I would structure the company like this:

PREPGRADE
|
ASSESSMENT CLOUD
|



The first version only needs:

Questions + Exams + CBT + Results + Basic Analytics.

Everything above that is the future.

61. YOUR PERSONAL EXECUTION PLAN

Given your broader goal of building a serious technology company, I would **not** make PrepGrade your only source of income immediately.

Use a two-engine strategy:

Engine 1 — Cash Flow

Your software/AI services business.

Use it to generate:

- client revenue;
- technical experience;
- relationships;
- capital.

Engine 2 — Asset

PrepGrade.

Invest your time and a controlled portion of profits into the product.

That gives you:

Cash-flow business → funds scalable product.

This is considerably safer than putting all your available money into PrepGrade before validation.

62. THE CAPITAL GATES

This is the rule I would personally impose.

Gate 1

No major spending before customer validation.

Gate 2

No major hiring before MVP usage.

Gate 3

No aggressive marketing before retention.

Gate 4

No expensive AI infrastructure before AI demand.

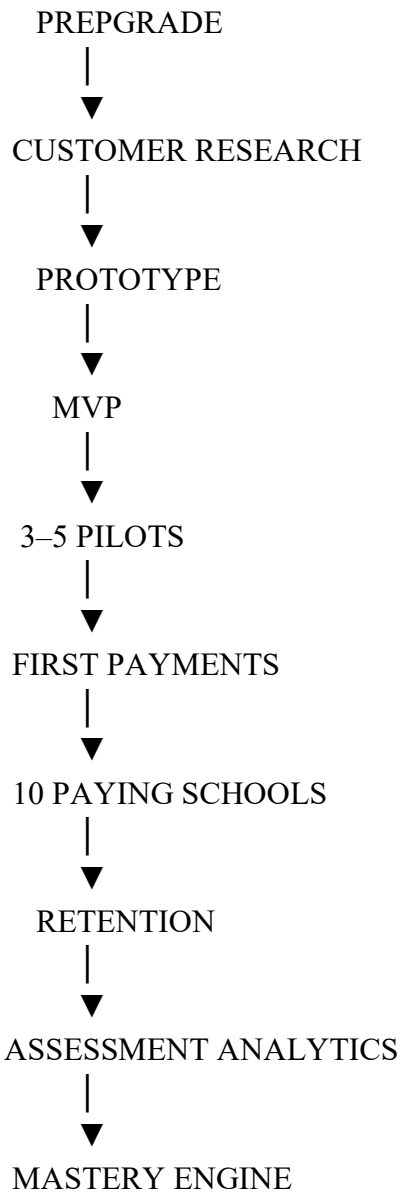
Gate 5

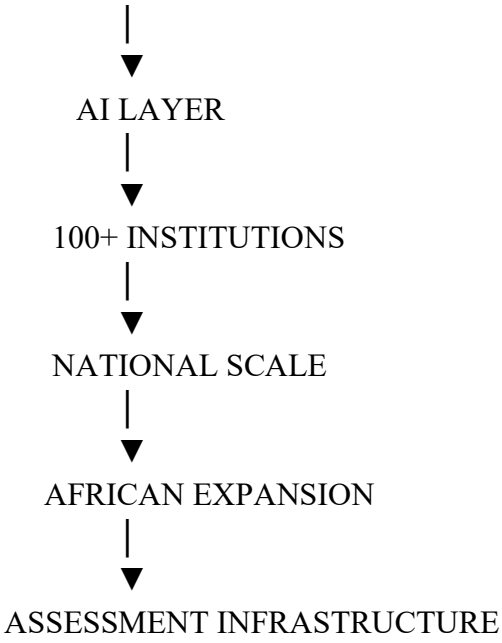
No national expansion before a repeatable sales process.

Gate 6

No international expansion before Nigerian product-market fit.

63. THE FINAL STRATEGIC ROADMAP





The immediate target is not ₦1 billion.

It is not 1 million students.

It is not even 1,000 schools.

Your first target is:

3 institutions using PrepGrade successfully.

Then:

10 institutions paying.

Then:

Those institutions staying.

Then:

Build the technology that makes the product increasingly difficult to replace.

That is how I would manage PrepGrade as a professional technology company rather than as a side-project.

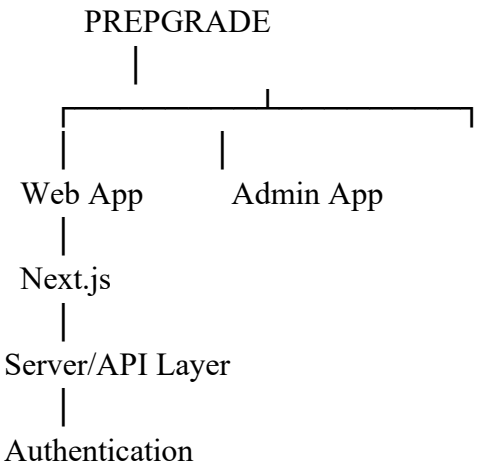
PREPGRADE — MVP BUILD SPECIFICATION

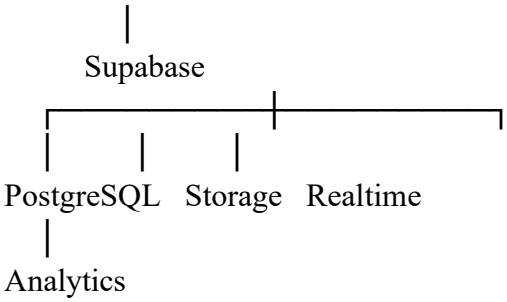
From PRD to Actual Software

We now move from **what PrepGrade should do** to **exactly how the first production-ready MVP should be built**.

1. TECHNICAL ARCHITECTURE

For the first version, I would deliberately keep the architecture simple.





Recommended initial stack

Layer	Technology
Frontend	Next.js + React + TypeScript
Styling	Tailwind CSS
UI	shadcn/ui or equivalent component system
Backend	Next.js server/API + Supabase
Database	PostgreSQL
Authentication	Supabase Auth or Clerk
File storage	Supabase Storage
Hosting	Vercel or equivalent
Monitoring	Sentry/equivalent
Analytics	PostHog/equivalent
Source control	GitHub
CI/CD	GitHub Actions
Design	Figma

The exact vendors can change. The architectural principles matter more.

2. WHY THIS STACK

You don't need:

- Kubernetes;
- microservices;
- five databases;
- a dedicated AI cluster;
- complicated DevOps;
- expensive infrastructure.

Not yet.

The MVP needs:

speed + reliability + security + maintainability.

A well-designed modular monolith can take PrepGrade very far before microservices become necessary.

3. APPLICATION STRUCTURE

I would structure the project approximately like:

```
prepgrade/
│
├── app/
│   ├── (marketing)/
│   │
│   ├── auth/
│   │   ├── login/
│   │   ├── register/
│   │   └── forgot-password/
│   │
│   ├── dashboard/
│   │   ├── student/
│   │   ├── teacher/
│   │   └── school/
│   │
│   ├── exams/
│   │   └── [examId]/
│   │
│   ├── attempts/
│   │   └── [attemptId]/
│   │
│   ├── results/
│   │   └── [resultId]/
│   │
│   └── admin/
│
├── components/
│   ├── ui/
│   ├── dashboard/
│   ├── exams/
│   ├── questions/
│   ├── analytics/
```

```
│   └── forms/
│
├── lib/
│   ├── auth/
│   ├── db/
│   ├── permissions/
│   ├── analytics/
│   └── validation/
│
├── services/
│   ├── exams/
│   ├── grading/
│   ├── analytics/
│   └── institutions/
│
├── types/
│
├── tests/
│
└── config/
```

Keep business logic out of random UI components.

4. DOMAIN MODULES

Internally, PrepGrade should be divided into domains:

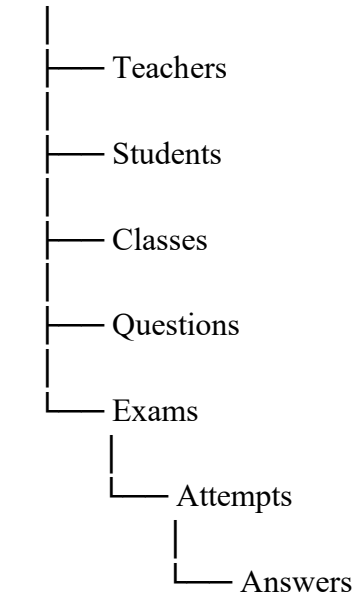
AUTH
INSTITUTIONS
USERS
STUDENTS
TEACHERS
CLASSES
QUESTIONS
EXAMS
ATTEMPTS
GRADING
RESULTS
ANALYTICS
NOTIFICATIONS

BILLING
AUDIT

Only the first nine or ten are essential for MVP.

5. DATABASE RELATIONSHIP

The core relationship is:



6. MULTI-TENANT RULE

Every institution-owned resource must have a clear ownership path.

For example:

exam.institution_id
question.institution_id
student.institution_id
teacher.institution_id
class.institution_id

Never depend on:

"The frontend won't show it."

The database must enforce it.

7. DATABASE SECURITY

If Supabase is used, enable:

Row Level Security

Example conceptual policy:

Teacher can read questions
WHERE
question.institution_id
=
teacher.institution_id

Student:

Can read attempt
WHERE
attempt.student_id
=
authenticated_user.student_id

Institution admin:

Can read users
WHERE
users.institution_id
=
admin.institution_id

Super admin has controlled platform-level access.

8. AUTHENTICATION ARCHITECTURE

Authentication identifies:

Who are you?

Authorization determines:

What are you allowed to do?

Never confuse the two.

Example:

User
↓
Authenticated
↓
Role = Teacher
↓
Institution = School A
↓
Permission = Create Exam
↓
ALLOW

9. INSTITUTION CREATION

Institution registration:

POST /institutions

Creates:

institution
institution_membership
admin profile

Transactionally.

If something fails, don't leave half-created accounts.

10. STUDENT CREATION

Initially support two methods.

Method 1

Admin manually creates student.

Method 2

CSV import.

CSV:

first_name,last_name,student_id,class,email
Ahmad,Sani,PG001,SS3A,ahmad@example.com

System:

Upload
↓
Validate
↓
Preview
↓
Confirm
↓
Create

11. CSV VALIDATION

Before import, detect:

- duplicate student ID;
- invalid email;
- missing class;
- missing name;
- duplicate row;
- invalid characters where relevant.

Show:

482 valid records

11 records require correction

Do not silently import bad data.

12. CLASS MANAGEMENT

Institution admin creates:

SS1 A
SS1 B
SS2 A
SS2 B
SS3 A
SS3 B

Each class belongs to:

institution
academic_session

Example:

2026/2027 — SS3A

This matters because schools repeat class names every academic year.

13. ACADEMIC SESSION

Make academic session a first-class entity.

Example:

2026/2027

Then:

institution
↓
academic session
↓
classes
↓
students

↓
assessments

This prevents historical data becoming messy.

14. QUESTION ARCHITECTURE

MVP should support:

Multiple Choice Questions

Example:

Question:
What is 2 + 2?

A. 2
B. 3
C. 4
D. 5

Database stores:

question
├── option A
├── option B
├── option C
└── option D

Correct answer:

C

15. QUESTION VERSIONING

This becomes important later.

Suppose a question is used in 10 exams.

Teacher changes the wording.

You shouldn't unexpectedly change historical exam results.

Therefore eventually:

Question
↓
Question Version
↓
Exam Snapshot

For MVP, when an exam is published, store enough immutable question information to preserve historical integrity.

16. EXAM CREATION

The exam builder should follow a wizard.

Step 1

Basic information.

Step 2

Questions.

Step 3

Settings.

Step 4

Assignment.

Step 5

Preview.

Step 6

Publish.

This is much better than a giant complicated form.

17. EXAM PREVIEW

Before publishing:

Preview as Student

This is mandatory.

Teacher should see exactly what students will see.

Then:

Back to Edit

or

Publish Exam

18. PUBLISHING RULE

Once students begin attempts:

Do not allow dangerous modifications.

For example, don't allow:

Delete Question 7.

Instead:

Create new exam version.

This protects examination integrity.

19. EXAM ASSIGNMENT

Support:

Assign to class

Most common.

Assign to individual students

Useful for:

- make-up tests;
- special assessments;
- remedial students.

20. EXAM ACCESS

Student sees:

Available

Exam can be started.

Upcoming

Exam exists but has not opened.

Completed

Student has submitted.

Expired

Exam window has closed.

21. ATTEMPT STATE MACHINE

This should be explicit.

NOT_STARTED
↓
IN_PROGRESS
↓
SUBMITTED
↓
GRADED

Exceptional states:

ABANDONED
EXPIRED
CANCELLED

This prevents messy logic.

22. START EXAM

When student clicks:

Start Exam

backend:

1. Verify assignment.
2. Verify exam is active.
3. Verify attempt limit.
4. Create attempt.
5. Set server start time.
6. Calculate end time.
7. Return exam payload.

Then frontend begins the experience.

23. TIMER

Store:

started_at
expires_at

Example:

started_at:
2026-10-08 18:00

expires_at:
2026-10-08 19:00

The browser displays remaining time.

The backend remains authoritative.

24. ANSWER SAVING

When student chooses option C:

POST /attempts/:id/answers

Payload conceptually:

question_id
selected_option_id
timestamp

Server validates:

- attempt exists;
- student owns attempt;
- attempt is active;
- question belongs to exam.

Then saves.

25. AUTOSAVE STRATEGY

Don't make a server request every millisecond.

Use a hybrid strategy:

Local

Immediately store current answer locally.

Server

Synchronize shortly afterward.

Periodic

Synchronize all unsynced answers.

This balances reliability and network efficiency.

26. SUBMISSION

When submitted:

POST /attempts/:id/submit

Backend:

1. Verify ownership.
2. Verify attempt status.
3. Verify exam state.
4. Lock attempt.
5. Finalize answers.
6. Calculate score.
7. Generate result.
8. Generate performance metrics.
9. Mark GRADED.

27. GRADING ENGINE

For each question:

selected_answer == correct_answer

Then:

correct → marks awarded
wrong → zero
unanswered → zero

Future versions can support:

- negative marking;
- partial credit;
- multiple correct answers;
- essays.

Not MVP.

28. RESULT CALCULATION

Example:

50 questions.

Each = 1 mark.

Student:

38 correct.

Then:

score = 38
maximum = 50
percentage = 76%

Store the result.

Do not repeatedly calculate everything from scratch when displaying the result.

29. PERFORMANCE ENGINE

After grading:

Attempt
↓
Answers
↓
Questions
↓
Topics
↓
Aggregate

Example:

Student answered:

Algebra:
12/15

Geometry:
8/12

Probability:
4/10

Performance engine generates:

Algebra: 80%
Geometry: 67%
Probability: 40%

30. PERFORMANCE HISTORY

Student should see:

Exam 1 61%
Exam 2 68%

Exam 3 72%
Exam 4 76%

Then chart:

Performance trend ↑

This is extremely motivating.

31. TEACHER CLASS ANALYTICS

Aggregate students:

Class Average
Median
Highest
Lowest
Pass Rate
Topic Accuracy

Median is important because average alone can be misleading.

32. INSTITUTION ANALYTICS

Institution dashboard:

Total students
Active students
Total teachers
Exams this month
Average performance
Completion rate

Then:

Academic Performance

Subject → Class → Topic.

33. EXPORTS

Initial exports:

CSV

Easy and cheap.

PDF

Later.

Excel

Later if customers strongly request it.

Do not spend weeks building a sophisticated reporting engine before users ask for it.

34. AUDIT LOGS

Important actions:

Created exam
Published exam
Deleted question
Added student
Removed student
Changed institution settings
Submitted exam
Changed permissions

Record:

user
action
resource
timestamp
metadata

This helps with security and dispute resolution.

35. DISPUTE HANDLING

Suppose a student says:

"I answered question 10 but it wasn't saved."

Admin should be able to investigate:

Attempt
↓
Question
↓
Answer events
↓
Timestamp
↓
Synchronization status

This is why logging matters.

36. ADMIN SUPPORT TOOL

Eventually internal PrepGrade staff need:

Search institution

Search user

View account status

View exam

View attempt

View system events

But:

Support staff should not casually modify student results.

Any privileged action should be audited.

37. QA STRATEGY

Testing should happen at three levels.

Unit tests

Test individual functions.

Integration tests

Test:

Exam creation → database → assignment.

End-to-end tests

Test:

Teacher creates exam → student takes exam → result appears.

38. CRITICAL E2E TEST

One automated test should reproduce:

Create institution
↓
Create teacher
↓
Create student
↓
Create question
↓
Create exam
↓
Assign exam
↓
Student starts exam

↓
Answers questions
↓
Submits
↓
System grades
↓
Teacher sees result

If this breaks, the release should be blocked.

39. CBT TESTING

Test:

Normal network

Works.

Slow network

Works.

Network interruption

Recovers.

Refresh

Attempt continues.

Browser closes

Attempt can recover.

Timer reaches zero

Attempt automatically submits.

Student opens two tabs

System handles safely.

Student attempts unauthorized exam

Rejected.

40. SECURITY TESTING

Test:

Student attempts teacher URL

→ 403/unauthorized.

Teacher attempts another institution

→ denied.

Institution A requests Institution B API data

→ denied.

Manipulated exam ID

→ denied.

Manipulated student ID

→ denied.

Expired authentication

→ rejected.

41. PERFORMANCE TEST

Before pilot, simulate:

100 concurrent students

taking an examination.

Then:

250

Then:

500

You need to know where the system begins degrading.

Do not wait for a real school examination to discover your infrastructure limit.

42. PILOT INFRASTRUCTURE

Initially, you don't need huge servers.

But you need:

- monitoring;
- backups;
- database protection;
- error tracking;
- rate limits;
- deployment rollback.

The goal is:

small infrastructure with professional engineering discipline.

43. DEPLOYMENT PIPELINE

Use:

Developer
↓
Git branch
↓
Pull Request
↓
Code Review
↓
Automated Tests
↓
Preview
↓
QA
↓
Production

Never make production the developer's playground.

44. ENVIRONMENTS

Maintain:

Development

Developer environment.

Staging

Production-like testing.

Production

Real customers.

Do not test experimental code directly on production.

45. BACKUP STRATEGY

At minimum:

- automated database backups;
- recovery procedure;
- documented backup retention;
- periodic recovery testing.

A backup that has never been tested is not a complete disaster-recovery strategy.

46. RELEASE STRATEGY

Initially:

Weekly releases

Only tested features.

For critical fixes:

Hotfix release.

Eventually:

- feature flags;
- gradual rollouts;
- rollback.

47. FEATURE FLAGS

Useful for:

AI_QUESTION_GENERATOR
ADVANCED_ANALYTICS
NEW_CBT_ENGINE
BETA_REPORTS

You can enable features for:

Pilot School A

without exposing them to everyone.

48. PRODUCT ANALYTICS

Track:

institution_created
teacher_invited
student_imported
question_created
exam_created
exam_published
exam_started
answer_saved
exam_submitted
result_viewed
analytics_viewed

Then build funnels.

49. ACTIVATION METRIC

Define:

Activated Institution

An institution is activated when:

It creates at least one exam and has at least one student successfully complete it.

This is much more meaningful than:

Registered institution.

50. RETENTION METRIC

A school is retained when it conducts another meaningful assessment within the defined measurement period.

For example:

30-day institutional assessment retention.

Don't measure only login retention.

51. NORTH STAR METRIC

Still:

Meaningful Assessments Completed

But add secondary metrics:

- assessments per institution;
- students assessed;
- teacher weekly activity;
- institution retention;
- paid conversion.

52. FIRST PRODUCT DASHBOARD

Your internal CEO dashboard should look roughly like:

PREPGRADE — COMPANY DASHBOARD

Institutions
12

Paying
7

Students
2,840

Teachers
164

Assessments
438

Attempts
28,492

30-Day Retention
78%

MRR
~~₦~~XXX,XXX

Critical Bugs
0

The actual numbers come from the business.

Never manufacture them for presentations.

53. CUSTOMER HEALTH DASHBOARD

Institution	Usage	Status
School A	High	🟢
School B	High	🟢
School C	Medium	🟡
School D	Low	🔴

Then customer success focuses on yellow/red accounts.

54. PRODUCT ROADMAP GOVERNANCE

Every month:

KEEP

Features working well.

IMPROVE

Features with problems.

REMOVE

Features nobody uses.

BUILD

Features with strong evidence.

This prevents product bloat.

55. THE FIRST 5 PILOT INSTITUTIONS

Do not choose them randomly.

Score each candidate:

Criterion	Weight
Student volume	20%
Examination frequency	20%
Decision-maker access	20%
Technology readiness	15%
Feedback willingness	15%
Payment potential	10%

Select institutions with the highest combined score.

56. PILOT SUCCESS SCORE

Each institution receives:

Adoption
Teacher Usage
Student Usage
Assessment Frequency
Completion
Feedback
Renewal Intent

Then:

80–100

Strong.

60–79

Needs improvement.

<60

At risk.

Again, this is an internal management framework, not an industry-standard benchmark.

57. PILOT REVIEW MEETING

Every two weeks with each serious pilot:

Ask:

1. What worked?
2. What failed?
3. What was confusing?
4. What saved time?
5. What do teachers want?
6. What do students want?

7. Would you pay?
8. Would you recommend PrepGrade?

Then classify feedback:

Bug
UX problem
Feature request
Training issue
Strategic opportunity

58. THE MOST IMPORTANT QUESTION

At the end of the pilot:

"If PrepGrade disappeared tomorrow, what would you miss?"

If they say:

"Nothing."

You have a problem.

If they say:

"We would have to go back to manual grading."

Excellent.

If they say:

"Our teachers depend on the analytics now."

Even better.

59. WHEN TO HIRE

Hire developer #2 when:

Current developers are becoming a bottleneck.

Hire QA when:

Release frequency and complexity make manual testing risky.

Hire customer success when:

Founder cannot personally support every customer.

Hire salesperson when:

You have a repeatable sales message.

Hire marketing when:

You understand which audience converts.

Do not hire departments to make the company "look big."

60. WHEN TO RAISE MONEY

I would consider external funding after evidence such as:

- functioning product;
- paying institutions;
- recurring revenue;
- retention;
- repeatable acquisition;
- clear market opportunity.

Then funding can accelerate:

- engineering;
- sales;
- content;

- security;
- expansion.

61. FUNDING MISTAKE TO AVOID

Don't raise:

₦50m

and suddenly spend:

- ₦10m office;
- ₦10m cars;
- ₦10m salaries;
- ₦5m branding;
- ₦5m events.

while the product has five customers.

Capital should increase the company's **capacity to produce customer value**.

62. THE PREPGRADE MONEY HIERARCHY

Revenue should initially go approximately toward:

Customer Value
↓
Product
↓
Reliability
↓
Customer Success
↓
Sales
↓
Team
↓
Expansion

Not:

Office
↓
Luxury
↓
Advertising
↓
Hope

63. BUSINESS MOAT ROADMAP

Year 1

Relationships.

Year 2

Workflow integration.

Year 3

Assessment data.

Year 4

Mastery algorithms.

Year 5

AI + ecosystem.

This is how the competitive moat compounds.

64. THE LONG-TERM PREPGRADE DATA MODEL

Eventually:

Institution
↓
Student
↓
Assessment
↓
Question
↓
Response
↓
Performance
↓
Topic Mastery
↓
Learning Recommendation
↓
New Assessment

Every cycle makes the system smarter.

65. THE REAL PRODUCT

The visible product is:

CBT.

But the real product becomes:

Understanding student performance.

And the eventual moat becomes:

Understanding how students learn and where assessment interventions work.

That is much harder to copy.

66. PREPGRADE'S THREE PRODUCT LAYERS

Layer 1 — Infrastructure

Questions, exams, CBT, grading.

Layer 2 — Intelligence

Analytics, mastery, diagnostics.

Layer 3 — Action

Practice, recommendations, AI, intervention.

The MVP only needs Layer 1 + a small part of Layer 2.

67. THE ULTIMATE USER EXPERIENCE

Eventually:

Teacher

"My SS2 class is weak in probability."

PrepGrade:

"47% average accuracy."

Teacher:

"Create revision assessment."

PrepGrade:

Creates a draft based on the weak concepts.

Teacher:

Reviews and publishes.

Students:

Take it.

PrepGrade:

Measures improvement.

Teacher:

Sees whether intervention worked.

That is the complete assessment loop.

68. PREPGRADE'S LONG-TERM COMPETITIVE POSITION

Do not position it as:

"Another JAMB app."

Do not position it as:

"Another school management system."

Position it as:

The assessment intelligence platform for African education.

That is the category I would attempt to own.

69. THE NEXT 30 DAYS — FOUNDER CHECKLIST

WEEK 1

☐ Finalise MVP scope

☐ Write product requirements

☐ Create competitor matrix

☐ Interview teachers

☐ Interview school administrators

☐ Interview students

☐ Create pilot prospect list

WEEK 2

☐ Finalise UI architecture

☐ Design student dashboard

☐ Design teacher dashboard

☐ Design school dashboard

☐ Design CBT

☐ Finalise database

☐ Finalise security model

WEEK 3

☐ Build authentication

☐ Build institutions

☐ Build users

☐ Build classes

☐ Build question bank

WEEK 4

☐ Build exam builder

☐ Build CBT

☐ Build grading

☐ Build results

☐ Prepare first pilot

70. YOUR FIRST 90-DAY KPI BOARD

Product

☐ MVP completed

☐ Critical bugs resolved

☐ CBT reliability validated

☐ Analytics working

Customers

☐ 30 discovery interviews

☐ 10 demos

☐ 5 pilot institutions

☐ 3 active institutions

Business

☐ First paying customer

☐ First recurring revenue

☐ First renewal

Learning

☐ 100+ pieces of customer feedback

☐ Top 10 product problems identified

☐ Product roadmap revised based on evidence

71. THE DECISION TREE

At day 90:

If users love it + don't pay

→ Fix monetization/value proposition.

If users pay + don't retain

→ Fix product value.

If users retain + acquisition is difficult

→ Fix distribution.

If acquisition + retention work

→ Scale.

If nobody cares

→ Pivot.

This is how professional product management works.

72. THE COMPANY SHOULD NEVER LOSE THIS DISCIPLINE

Every quarter ask:

What evidence do we have that PrepGrade is becoming more valuable?

Evidence should come from:

- usage;
- retention;
- payments;
- referrals;
- assessment volume;
- measurable academic/operational value.

Not from:

- followers;
- likes;
- beautiful screenshots;
- registered users who never return.

73. FINAL EXECUTION MODEL

The entire company can be reduced to this:

DISCOVER
↓
VALIDATE
↓
BUILD
↓
PILOT
↓
MEASURE
↓
IMPROVE

↓
MONETIZE
↓
RETAIN
↓
SCALE
↓
EXPAND

And the capital discipline is:

Evidence
↓
Small investment
↓
More evidence
↓
Larger investment
↓
More evidence
↓
Scale

PREPGRADE'S FIRST MISSION

Don't build a giant company.

Don't build 100 features.

Don't spend millions.

Build one thing exceptionally well:

Allow an institution to create an examination, allow students to take it reliably, automatically grade it, and immediately show teachers where students need help.

Then get **3 schools** to love it.

Then **10 schools to pay**.

Then make those 10 stay.

Once that happens, you have earned the right to build the bigger PrepGrade vision.

PREPGRADE — ENGINEERING BLUEPRINT

Database → Security → API → Codebase → CI/CD → QA

Now we turn the product specification into an **engineering execution document** that a professional development team can actually work from.

1. ENGINEERING OBJECTIVE

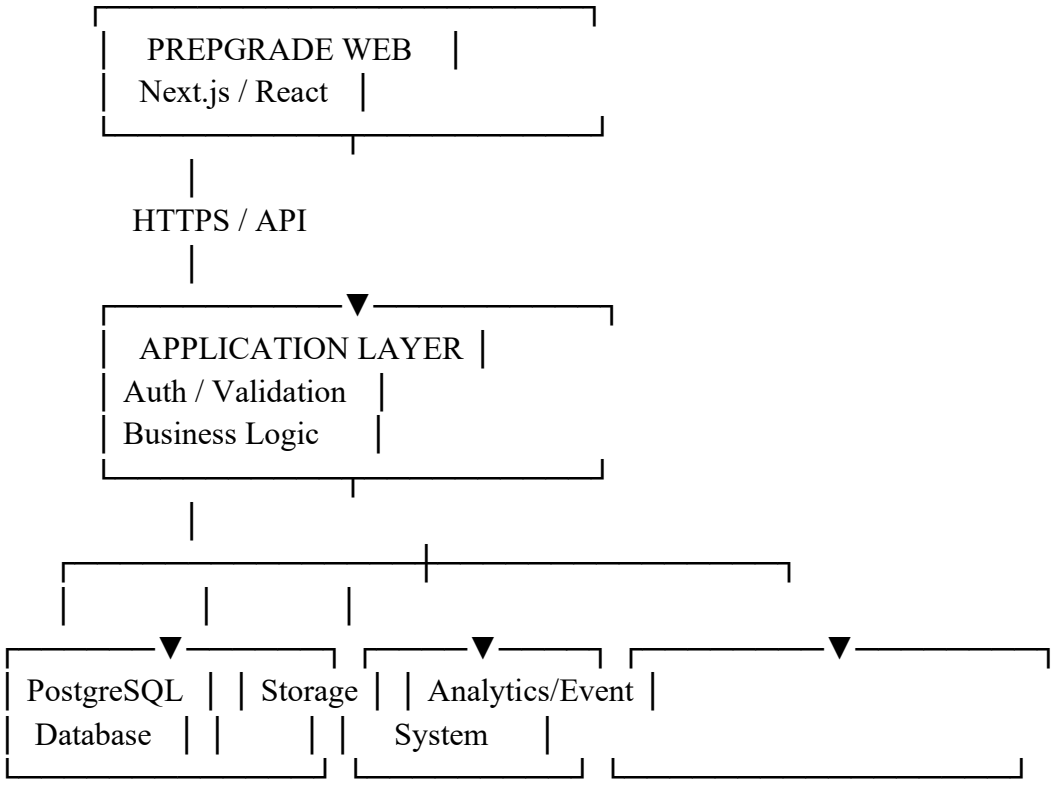
The first production MVP must allow this complete flow:

Institution → Teacher → Question → Exam → Student → CBT → Submission → Grading → Analytics

If that flow works reliably, PrepGrade has its technical foundation.

Everything else is secondary.

2. SYSTEM ARCHITECTURE



For MVP, keep this architecture as a **modular monolith**.

Do not introduce microservices yet.

3. CORE TECHNOLOGY STACK

Frontend

Next.js + React + TypeScript

Why:

- excellent web application ecosystem;
- strong routing;
- server-side capabilities;
- good developer availability;
- suitable for dashboards;
- scalable enough for the MVP.

Styling

Tailwind CSS

With a reusable component system.

Database

PostgreSQL

Because PrepGrade is fundamentally a relational-data product.

Relationships matter:

Institution
→ Classes
→ Students
→ Exams
→ Questions
→ Attempts
→ Results

PostgreSQL is an excellent fit.

4. AUTHENTICATION

You have two reasonable options:

Option A

Supabase Auth.

Option B

Clerk + Supabase PostgreSQL.

For a lean MVP, I would favour:

Supabase Auth + PostgreSQL + RLS

because it reduces architectural complexity.

Clerk can be reconsidered if the product's authentication requirements become more sophisticated.

5. STORAGE

Use object storage for:

- institution logos;
- student profile images if eventually permitted;
- report files;
- imported CSVs;
- other controlled documents.

Never put large files directly inside PostgreSQL.

6. DATABASE SCHEMA

Here is the core production model.

profiles

id
full_name
email
phone
avatar_url
created_at
updated_at

This represents the application-level user profile.

7. institutions

id
name
slug
logo_url
institution_type
status
owner_id
created_at
updated_at

Example:

id: UUID
name: Example Academy
slug: example-academy
type: secondary_school
status: active

8. institution_members

id
institution_id
user_id
role
status
created_at

Possible roles:

SUPER_ADMIN
INSTITUTION_ADMIN
TEACHER
STUDENT

This table becomes the foundation of multi-tenancy.

9. academic_sessions

id
institution_id
name
start_date
end_date
status
created_at

Example:

2026/2027

10. classes

id
institution_id
academic_session_id
name
level
created_at

Example:

SS3A
SS3B
SS2A

11. students

id
institution_id
user_id
student_code
class_id
created_at
updated_at

A student belongs to an institution.

12. teachers

id
institution_id
user_id
employee_code
created_at
updated_at

13. subjects

Initially, PrepGrade can maintain a platform-level subject catalogue.

id
name
code
status

Examples:

MAT — Mathematics
ENG — English
PHY — Physics
CHE — Chemistry
BIO — Biology

14. topics

id
subject_id
name
status
created_at

Example:

Subject:
Mathematics

Topic:
Probability

15. questions

id
institution_id
subject_id
topic_id
created_by
question_text
question_type
difficulty
explanation
source
status
created_at
updated_at

MVP question types:

MCQ_SINGLE

Future:

MCQ_MULTIPLE
TRUE_FALSE
SHORT_ANSWER
ESSAY

16. question_options

id
question_id
option_key
option_text

is_correct
created_at

Example:

A → 10
B → 15
C → 20
D → 25

Only one option can be correct for:

MCQ_SINGLE

17. exams

id
institution_id
created_by
subject_id
title
description
duration_minutes
attempts_allowed
randomize_questions
randomize_options
show_result_immediately
start_at
end_at
status
created_at
updated_at

18. exam_questions

id
exam_id
question_id
position

marks
created_at

This allows one question to be reused across different exams.

19. exam_assignments

id
exam_id
class_id
student_id
assigned_at
status

Allow either:

- class assignment;
- individual assignment.

20. exam_attempts

id
exam_id
student_id
attempt_number
started_at
expires_at
submitted_at
status
score
max_score
percentage
time_used_seconds
created_at
updated_at

This is one of the most important tables in PrepGrade.

21. attempt_answers

id
attempt_id
question_id
selected_option_id
is_correct
marks_awarded
answered_at
updated_at

Never overwrite the entire attempt when a student changes one answer.

Update the individual answer.

22. performance_metrics

For MVP:

id
student_id
attempt_id
subject_id
topic_id
questions_attempted
correct_answers
accuracy
created_at

This provides the raw material for future analytics.

23. audit_logs

id
institution_id
user_id
action
entity_type

entity_id
metadata
ip_address
user_agent
created_at

Use this for sensitive operations.

24. subscriptions

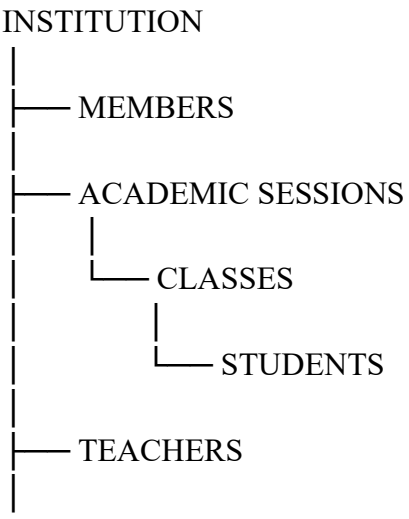
Even if billing isn't active on day one, design the architecture for it.

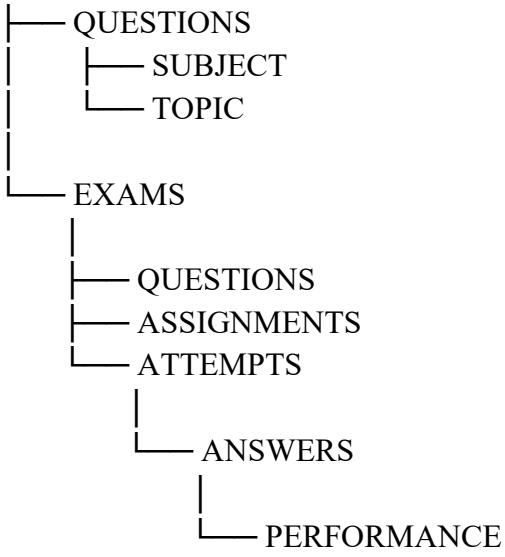
id
institution_id
plan_id
status
started_at
expires_at
created_at
updated_at

But do **not** build a complicated billing engine before you have paying customers.

25. DATABASE RELATIONSHIP

The final conceptual relationship is:





26. TENANT SECURITY MODEL

Every request must be evaluated against:

Who?
↓
What institution?
↓
What role?
↓
What resource?
↓
What permission?

Example:

Teacher from:

School A

requests:

GET /questions

Database verifies:

question.institution_id
=
teacher.institution_id

If false:

Access denied.

27. RLS POLICY PRINCIPLE

Do not rely only on application code.

Use database-level policies.

Conceptually:

```
CREATE POLICY "Users can view own institution data"
ON questions
FOR SELECT
USING (
   institution_id = get_user_institution_id()
);
```

The exact implementation should be tested carefully with Supabase's current RLS semantics.

28. STUDENT SECURITY

A student must only be able to retrieve:

- their own attempts;
- their own answers;
- their own results;
- exams assigned to them.

A malicious request such as:

/results/student/another-student-id

must return:

403 / not found, depending on the security design.

Never expose another student's academic record.

29. TEACHER SECURITY

A teacher can see:

- their classes;
- students in permitted classes;
- exams they own/manage;
- results for their assigned classes.

A teacher should not automatically see:

Every student in the entire platform.

30. INSTITUTION ADMIN SECURITY

Institution admins can see their institution.

They cannot:

- access another school;
- alter PrepGrade system configuration;
- modify platform-level billing rules;
- view another tenant's database records.

31. SUPER ADMIN

Super admin should be extremely restricted.

Use:

- MFA;
- strong authentication;
- audit logging;
- limited personnel;
- privileged action monitoring.

A platform administrator is one of the highest-risk accounts.

32. API ARCHITECTURE

Group APIs by domain:

/api/auth
/api/institutions
/api/students
/api/teachers
/api/classes
/api/questions
/api/exams
/api/attempts
/api/results
/api/analytics
/api/admin

Don't create one giant:

/api

handler.

33. QUESTION API

Create

POST /api/questions

List

GET /api/questions

Retrieve

GET /api/questions/:id

Update

PATCH /api/questions/:id

Archive

POST /api/questions/:id/archive

Avoid hard deletion for important assessment content.

34. EXAM API

POST /api/exams
GET /api/exams
GET /api/exams/:id
PATCH /api/exams/:id
POST /api/exams/:id/publish
POST /api/exams/:id/assign

35. ATTEMPT API

POST /api/exams/:id/start
GET /api/attempts/:id
PUT /api/attempts/:id/answers/:questionId
POST /api/attempts/:id/sync
POST /api/attempts/:id/submit

36. IMPORTANT API RULE

Never trust the frontend.

For example:

The frontend sends:

```
{
  "score": 100
}
```

The backend should **ignore that score**.

The server calculates the score from the submitted answers.

Otherwise students could manipulate results.

37. RESULT GENERATION

Submission pipeline:

Student submits
↓
Validate attempt
↓
Lock attempt
↓
Fetch exam snapshot
↓
Grade answers
↓
Calculate score
↓
Calculate percentage
↓
Calculate topic metrics
↓
Store result
↓
Mark GRADED
↓
Return result

This should be an atomic/transactional operation wherever practical.

38. EXAM SNAPSHOT

This is important enough to emphasize.

Suppose:

Exam A

uses Question 12.

Tomorrow the teacher edits Question 12.

Historical Exam A should still represent what students actually saw.

Therefore, published exams should have an immutable question snapshot/version.

Conceptually:

Question Bank
↓
Publish Exam
↓
Create Exam Question Snapshot
↓
Student sees snapshot

This prevents historical results from changing.

39. CBT RELIABILITY

The CBT engine is the highest-risk MVP component.

The system must handle:

Refresh

Student can recover attempt.

Connection loss

Unsynced answers are preserved locally.

Browser close

Attempt remains available.

Multiple tabs

Prevent conflicting attempts where appropriate.

Expiration

Server ends the attempt.

Duplicate submission

Second submission should not alter the final result.

40. LOCAL CBT STATE

Conceptually:

```
Browser
│
├── Current question
├── Selected answers
├── Marked questions
└── Sync queue
```

Then:

```
Sync queue
↓
Server
↓
Database
```

This is especially useful for Nigerian environments where mobile connectivity may be inconsistent.

41. LOW-BANDWIDTH DESIGN

Avoid:

- giant images;
- unnecessary animations;
- huge JavaScript bundles;
- video on CBT pages;
- excessive API requests.

The CBT interface should be lightweight.

The exam should fundamentally be:

text + options + state.

42. MOBILE-FIRST CBT

Prioritize:

Smartphone

before:

Desktop.

The student should be able to comfortably:

- read question;
- select answer;
- navigate;
- see timer;
- submit.

43. EXAM SECURITY

For MVP, don't attempt invasive surveillance.

Instead focus on:

- secure login;
- controlled exam windows;
- attempt limits;
- server-side timing;
- randomized questions;
- randomized options;
- audit logs;
- secure result calculation.

Later, institutions can request more advanced examination controls.

44. RANDOMIZATION

If enabled:

Questions

Randomize presentation order.

Options

Randomize answer choices.

Important:

The system must maintain the mapping:

Displayed option
↓
Original option
↓
Correctness

Otherwise grading errors can occur.

45. QUESTION DIFFICULTY

Initially:

Teacher-selected:

Easy
Medium
Hard

Later, calculate empirical difficulty from actual student performance.

Example:

If 90% of students answer correctly:

likely easy.

If 30%:

likely difficult.

But don't automatically change difficulty labels without proper assessment methodology.

46. FUTURE QUESTION INTELLIGENCE

Eventually store:

Question
↓
Attempts
↓
Accuracy
↓
Average Time
↓
Distractor Performance
↓
Difficulty Estimate

This can become valuable assessment science.

47. CI/CD

Every pull request:

Git Push
↓
Lint
↓
Type Check

↓
Unit Tests
↓
Integration Tests
↓
Build
↓
Preview Deployment

Only after success:

merge.

Production deployment should happen through controlled CI/CD.

48. BRANCHING STRATEGY

Simple:

main
develop
feature/*
bugfix/*
hotfix/*

Or even:

main
feature/*
hotfix/*

for a small team.

Don't create complicated Git processes for a three-person team.

49. CODE REVIEW

Every production change should ideally have:

- one author;

- one reviewer.

Reviewer checks:

- correctness;
- security;
- performance;
- maintainability;
- tests.

50. ENVIRONMENT VARIABLES

Never commit secrets.

Example:

NEXT_PUBLIC_APP_URL
SUPABASE_URL
SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
SENTRY_DSN
ANALYTICS_KEY

Never place:

SUPABASE_SERVICE_ROLE_KEY

inside client-side code.

51. SECRET MANAGEMENT

Development:

.env.local

Production:

Secure hosting environment variables / secret manager.

Never:

- commit secrets;
- send secrets through WhatsApp;
- place API keys in screenshots;
- hard-code private credentials.

52. MONITORING

At minimum monitor:

Application

- errors;
- latency;
- crashes.

Database

- CPU;
- storage;
- connections;
- slow queries.

Exam engine

- failed starts;
- failed saves;
- failed submissions;
- grading errors.

53. CRITICAL ALERTS

These should trigger immediate attention:

Exam submission failure rate increasing.

Database unavailable.

Authentication outage.

Unexpected grading failures.

Major tenant-isolation error.

A failed marketing page is annoying.

A failed exam submission is a **business-critical incident**.

54. INCIDENT SEVERITY

SEV-1

Exam/data/security failure.

Immediate response.

SEV-2

Major feature unavailable.

Rapid response.

SEV-3

Non-critical bug.

Normal sprint.

SEV-4

Cosmetic/minor issue.

Backlog.

55. DISASTER RECOVERY

Document:

RPO

How much data can potentially be lost?

RTO

How quickly can service be restored?

For the MVP, establish realistic targets based on your infrastructure.

Then actually test restoration.

56. QA RELEASE CHECKLIST

Before every major release:

Authentication

☐ Register

☐ Login

☐ Logout

☐ Reset password

Institution

☐ Create institution

☐ Add teacher

☐ Add student

☐ Create class

Questions

☐ Create

☐ Edit

☐ Archive

☐ Search

Exams

☐ Create

☐ Preview

☐ Publish

☐ Assign

CBT

☐ Start

☐ Answer

☐ Navigate

☐ Autosave

☐ Submit

☐ Recover

Results

☐ Grade

☐ Display

☐ Analytics

Security

☐ Tenant isolation

☐ RBAC

☐ RLS

☐ Audit logs

57. ACCEPTANCE TEST — COMPLETE SYSTEM

The QA tester creates:

School A

Then:

Teacher A.

Student A.

Question:

What is 2 + 2?

Correct:

C.

Exam:

Mathematics Test.

Assign.

Student logs in.

Student starts.

Student selects C.

Student submits.

System calculates:

100%.

Teacher views result.

Teacher sees:

Student A — 100%.

Institution sees:

Mathematics average — 100%.

Then QA attempts to access the same result from **School B**.

Expected:

Access denied.

If all of that works:

core MVP flow passes.

58. DEVELOPMENT TEAM TICKETS

Sprint 1

Infrastructure

- Initialize repository.
- Configure TypeScript.
- Configure linting.
- Configure formatting.
- Configure CI.
- Configure environments.

Auth

- Registration.
- Login.
- Logout.
- Password reset.
- Role model.

Sprint 2

Institution

- Institution creation.
- Institution dashboard.

- Members.
- Academic sessions.
- Classes.

Sprint 3

Users

- Teacher management.
- Student management.
- CSV import.
- Invitations.

Sprint 4

Questions

- Question bank.
- Create question.
- Edit question.
- Archive.
- Subject/topic.

Sprint 5

Exams

- Exam builder.
- Add questions.
- Exam settings.
- Preview.
- Publish.
- Assignment.

Sprint 6

CBT

- Start attempt.
- Timer.
- Question navigation.
- Answer selection.
- Mark for review.
- Autosave.
- Recovery.
- Submission.

Sprint 7

Results

- Grading engine.
- Result page.
- Exam history.
- Performance metrics.

Sprint 8

Analytics

- Student analytics.
- Class analytics.
- Institution analytics.

Sprint 9

Security

- RLS.
- RBAC.
- Audit logs.

- Rate limiting.
- Security testing.

Sprint 10

Pilot hardening

- Performance.
- Mobile testing.
- Error handling.
- Monitoring.
- Backup verification.
- UX improvements.

59. DEFINITION OF MVP COMPLETE

The MVP is **not** complete when:

"All tickets are done."

It is complete when:

Technical

☐ Core flow works

☐ Security passes

☐ CBT is reliable

☐ Results are accurate

☐ Mobile works

☐ Monitoring works

Product

☐ Teacher can create exam

☐ Student can complete exam

☐ Institution can see performance

Business

☐ Pilot institutions onboarded

☐ Real assessments completed

☐ Feedback collected

60. PILOT LAUNCH CHECKLIST

Before allowing real students:

Product

☐ Landing page

☐ Authentication

☐ Institution dashboard

☐ Teacher dashboard

☐ Student dashboard

☐ CBT

☐ Results

Technical

☐ Production database

☐ Backups

☐ Monitoring

☐ Error tracking

☐ Security policies

☐ Domain

☐ SSL

Operations

☐ Support channel

☐ Pilot agreement

☐ Onboarding guide

☐ Emergency contact

☐ Feedback form

61. PILOT SUPPORT

Create one direct support channel.

For example:

PrepGrade Pilot Support

But do not allow support to become chaotic.

Categorize:

Technical
Account
Exam
Question
Billing
Feature Request

62. CUSTOMER FEEDBACK SYSTEM

After each assessment, ask teachers:

1–5

How easy was it to create this assessment?

1–5

How reliable was the examination?

1–5

How useful were the results?

Then:

What should we improve?

Now feedback becomes measurable.

63. FIRST PRODUCT BENCHMARKS

These are internal targets rather than promises:

Time to first exam

< 24 hours after onboarding.

Teacher exam creation

< 10 minutes.

Student exam start

< 1 minute.

Result generation

Near-immediate for ordinary objective exams.

Critical exam failures

As close to zero as practically possible.

Pilot institution retention

Aim for strong repeat usage.

64. THE MOST IMPORTANT KPI

After launch, watch:

Exams conducted per active institution per month.

Why?

Because:

A school conducting one exam and disappearing isn't a healthy customer.

A school conducting:

5 → 10 → 15 assessments

is demonstrating increasing dependence.

65. SECOND MOST IMPORTANT KPI

Percentage of institutions conducting another assessment.

This is your early retention signal.

66. THIRD

Paid conversion.

Because:

Usage proves value.

Payment proves commercial value.

Retention proves durable value.

67. FOURTH

Revenue per institution.

This tells you whether the business can eventually support itself.

68. FIFTH

Customer acquisition cost.

Once you have enough sales data:

CAC =
Sales + Marketing Costs
÷
New Paying Institutions

69. THE PREPGRADE GROWTH ENGINE

Eventually:

Sales
↓
Institutions
↓
Usage
↓
Results
↓
Value
↓
Retention
↓
Referrals
↓
More Sales

Then marketing accelerates it.

70. WHAT NOT TO DO

Do not:

Build a mobile app first.

The responsive web application can validate the product.

Build AI first.

You need assessment data.

Hire 15 developers.

You need evidence.

Spend millions on ads.

You don't yet know your best customer.

Build school ERP.

It distracts from your differentiation.

Add 100 exam subjects.

Start with your pilot requirements.

Build everything for JAMB/WAEC/NECO immediately.

Start with the assessment engine.

71. THE MOST IMPORTANT ARCHITECTURAL DECISION

Build the system so:

Exam types are configuration, not completely different products.

The same engine should eventually support:

School Test
JAMB Practice
WAEC Practice
NECO Practice
Tutorial Test
Mock Exam
Custom CBT

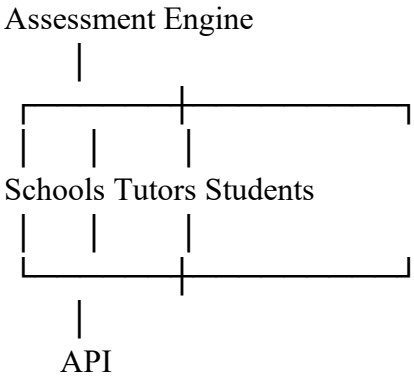
The difference should primarily be:

- content;
- rules;
- configuration;
- branding.

Not entirely separate codebases.

72. PREPGRADE AS A PLATFORM

This gives you:



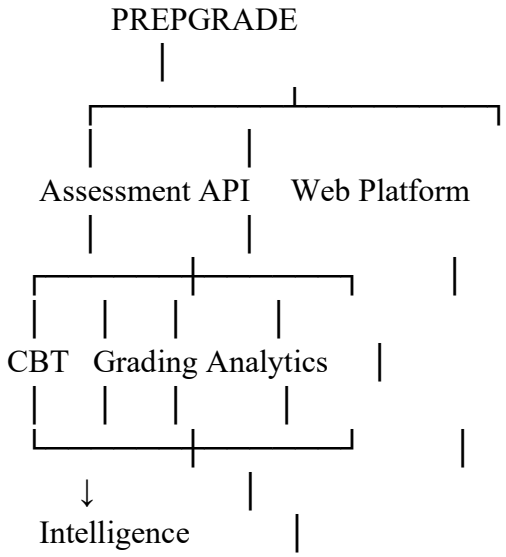
One core engine.

Many customers.

That is scalable.

73. LONG-TERM PRODUCT ARCHITECTURE

Eventually:



↓ |
AI Layer |
↓ |
Recommendations |

This is the long-term architecture.

But you don't build all of it now.

74. THE BUILDING PRINCIPLE

The technical team should always ask:

Can we solve this with the simplest reliable architecture?

If yes:

Do that.

Complexity is not sophistication.

75. THE FINAL MVP DEFINITION

If I were the CTO, I would officially define PrepGrade MVP as:

A secure multi-tenant web platform where educational institutions can manage teachers, students and classes; teachers can create and assign objective examinations from a structured question bank; students can take reliable timed CBT examinations; the system automatically grades attempts; and teachers/institutions can analyse student performance by score, subject and topic.

Anything outside that definition must justify itself.

76. THE NEXT BUILD STAGE

Once this MVP specification is approved, the next engineering artifacts should be produced in this order:

1.

Database ERD

2.

Complete SQL migration

3.

RLS security policies

4.

API contract

5.

Figma/UI design system

6.

Developer tickets

7.

Automated test plan

8.

Deployment configuration

9.

Pilot onboarding package

10.

Production launch checklist

That gives the development team a real execution path rather than another high-level startup document.

THE PREPGRADE ENGINEERING RULE

The first version should be boringly reliable.

Not overloaded with AI.

Not overloaded with animations.

Not overloaded with features.

A teacher creates an exam.

A student takes it.

The system doesn't lose the student's answers.

The system grades correctly.

The teacher immediately understands the result.

If PrepGrade can do those five things exceptionally well, you have the foundation of a serious company.